GITHUB COPILOT AND CHATGPT COMPARISON IN IMPROVING SOFTWARE DEVELOPMENT PRODUCTIVITY

Lappeenranta–Lahti University of Technology LUT
Master's Programme in Software Product Management and Business, Master's Thesis
2024
Kitta Kongas
Examiners: Assistant professor Dominik Siemon
Ram Gurung, M.Sc. (Tech.)

ABSTRACT

Lappeenranta–Lahti University of Technology LUT
LUT School of Engineering Science
Software Engineering

Kitta Kongas

GitHub Copilot and ChatGPT comparison in improving software development productivity

Master's thesis
2024

60 pages, 18 figures and 1 table

Examiners: Assistant Professor Dominik Siemon
Ram Gurung, M.Sc. (Tech.)

Keywords: Generative AI, Software engineering productivity, GitHub Copilot, ChatGPT

Generative AI tools have the potential to improve software development productivity. Many generative AI assistants are available that can help programmers solve their tasks faster and with less effort. Development productivity is an important aspect of software engineering. Productivity and satisfaction of developers are closely related. This thesis aims to compare two different types of generative AI tools: GitHub Copilot and ChatGPT, in improving software development productivity. GitHub Copilot is a tool connected to IDE and ChatGPT is a chatbot based on natural language interactions. The study was done by collecting information from literature and performing a case study at a company. This background information was viewed through the theoretical lenses of affordance theory and software development productivity theory. The result of the study was that GitHub Copilot is best suited to be used in acceleration mode when the developer already understands the problem to be solved. ChatGPT can be used in the ideation and architectural design phase in addition to programming tasks. Which tool brings the best benefits is context-dependent and the willingness and ability of the developer to use a specific tool is important for obtaining the productivity benefits. Including measurements for self-perceived productivity improvements alongside automatic measurements such as acceptance rate creates a more complete picture of the productivity improvements from generative AI assistants.

TIIVISTELMÄ

Lappeenrannan–Lahden teknillinen yliopisto LUT
Teknis-luonnontieteellinen
Tietotekniikka

Kitta Kongas

GitHub Copilotin ja ChatGPT:n vertailu ohjelmistokehityksen tuottavuuden parantamisessa

Tietotekniikan diplomityö
2024

60 sivua, 18 kuvaa ja 1 taulukko

Tarkastajat: Apulaisprofessori Dominik Siemon
Ram Gurung, DI

Avainsanat: Generatiivinen AI, ohjelmistokehityksen tuottavuus, ChatGPT, GitHub Copilot

Generatiivisella tekoälyllä on hyvät mahdollisuudet parantaa ohjelmistokehityksen tuottavuutta. Tarjolla on useita työkaluja, jotka voivat auttaa ohjelmoijia ratkaisemaan tehtävänsä nopeammin ja vaivattomammin. Kehityksen tuottavuus on tärkeä osa ohjelmistokehitystä. Tuottavuus ja kehittäjien tyytyväisyys ovat läheisessä suhteessa toisiinsa. Tämän työn tarkoituksena on vertailla kahta erityppistä GenAI -työkalua GitHub Copilotia ja ChatGPT:tä tuottavuuden parantamisessa. GitHub Copilot on IDEssä mukana oleva ohjelmoijan työkalu, kun taas ChatGPT on chatbot, jonka kanssa käyttäjä keskustelee luonnollisella kielellä. Tutkimus tehtiin kirjallisuuskatsauksella ja suorittamalla tapaustutkimus valitussa yrityksessä. Tätä tietoa tarkasteltiin teoreettisia viitekehyksiä kuten ohjelmistokehityksen tuottavuuskehityksen teoriaa vasten. Tutkimuksen lopputulos oli, että GitHub Copilot soveltuu parhaiten kiihdytysvaiheesen, kun kehittäjä ymmärtää ratkaistavan ongelman. ChatGPT soveltuu ohjelmointitehtävien lisäksi ideointiin ja arkkitehtuurisuunnittelun vaiheisiin. Työkalun tuomat hyödyt riippuvat kontekstista ja kehittäjän halu tärkeä kyky käyttää työkalua on tärkeää tuottavuushyötyjen saavuttamiseksi. Mittareina tuottavuuden parantamisessa kannattaa hyödyntää automaattimittausten lisäksi myös muita mittareita, kuten kehittäjien itse kokemaa tuottavuuden parantumista.

ACKNOWLEDGEMENTS

I would like to thank my supervisor Dominik Siemon for guiding me through the thesis process and especially in helping to refine the topic for this thesis. Also, I want to thank my family and especially my husband for supporting me in my studies and in writing the thesis. People at the case study company were very helpful, so I am grateful for all your support.

ABBREVIATIONS

LLM	Large language model
GenAI	Generative AI
AI	Artificial intelligence
IDE	Integrated development environment
RQ	Research question

Table of contents

Abstract

(Acknowledgements)

(Abbreviations)

Contents

1 Introduction

Generative AI tools have already changed how software developers solve problems and collaborate to build software (Ulfnes et al., 2024). Many tools may help programmers in solving their everyday problems. However, little information is available on how these tools should be best adopted and the concrete benefits and potential concerns. Developers may also need to change their working practices to gain benefits from tools (Pereira et al., 2024).

Development productivity is an important topic in software engineering and has been studied extensively (Forsgren et al., 2021). Productivity and developer satisfaction are closely related (Storey et al., 2019). Companies looking to adopt GenAI tools into their software engineering workflow are likely expecting productivity benefits. Experts have personal preferences regarding the tools and use them to make problem-solving efficient and more enjoyable. Following the affordance theory (Matei, 2020), experts may find different usage purposes for generative AI assistants based on their background.

Some of the most adopted tools in programming include ChatGPT and GitHub Copilot. ChatGPT is a chatbot based on artificial intelligence developed by OpenAI and it gains information from the internet-based text it was trained on (Komp-Leukkunen, 2024). GitHub Copilot on the other hand is an AI coding assistant that is particularly targeted to help in writing code with less effort and faster (GitHub, About GitHub Copilot). The tools support partly different software engineering use cases and when they support the same use cases, they do it differently (Pereira et al. 2024, 10).

To gain the best productivity benefits from the tools it should be understood how they differ, and which tool should be chosen to get the expected benefit. This thesis explores the two generative AI assistants ChatGPT and GitHub Copilot and the opportunities for productivity improvements.

1.1 Objectives and Research Questions

The main objective of this thesis is to investigate how generative AI solutions GitHub Copilot and ChatGPT can improve software development efficiency. As the tools support different types of use cases, the differences in how developers can utilize these tools in their daily work are examined. Also measuring the gained productivity improvements is researched, as it is important to understand how the positive impact can be verified.

The research questions of the thesis are the following:

- RQ1: What are the key differences in using GitHub Copilot and ChatGPT for improving software development productivity?
- RQ2: Which type of assistance brings the most productivity improvements?
- RQ3: How can the productivity improvements gained from using GenAI tools in software development be measured?

A literature review is conducted to reach the objectives and answer research questions. The literature review focuses on gathering information about the two generative AI tools and their differences, how developers use them to improve productivity, and what results are available from their application to software development. Also measuring the software development efficiency and measuring improvements gained by generative AI tools is examined. To gather enough academic background information for the study theoretical frameworks are included to give background information on what drives people to take digital solutions into use and how they utilize them. To further deepen the understanding of GenAI tool utilization and productivity improvements a case study is done in a large Finnish organization.

1.2 Research Methods

Two main paradigms in the information systems discipline are behavioral and design science. The behavioral science approach is focused on theories that explain or predict organizational or human behavior. The design-science approach on the other hand aims to

extend the organization or human boundaries by the creation of artifacts. These approaches are integral to information systems research that is centered around the crossing point of people, organizations and technology. (Hevner et al., 2024) This thesis takes a viewpoint of the behavioral science paradigm. The productivity benefits of generative AI assistants are seen from a technological view and how developers interact with the tools, adopt them and perceive their value. The affordance model and software development productivity theory are included as theoretical lenses for the study to strengthen this viewpoint. Selected research methods for the study are literature review and case study.

The literature review is conducted to gather existing information on the generative AI tools, their usage and their relation to software engineering productivity. The information is gathered mainly from two sources, LUT Primo and Google Scholar. The used search strings in the services used were the following: “Software development generative AI”, Software development GenAI”, “Software development productivity generative AI”, “ChatGPT software development” and “GitHub Copilot software development”. Also, snowballing was occasionally used as a search method, meaning that the references of the found articles were used to find more relevant material.

The comprehensive literature review gives a good foundation for the case study research. After this phase, it is possible to identify where more contributions are needed. An important output of the literary review phase is understanding the need for further investigation. (Verner et al., 2009) Case studies in software engineering can be defined as empirical research that investigates one instance of a phenomenon in a real-life context. Three main research strategies link to case studies: surveys, experiments, and action research. Case studies in software engineering tend to take an improvement approach like action research. (Runeson et al., 2012) Also, the theories give lenses for the case study. The targets and structure of the case study are refined in this thesis after the literature review. Surveys and interviews are used to collect data. The case study aims to confirm the hypotheses formed based on the theories and literature review section and to give actionable insights for the case company and organizations in similar settings.

1.3 Structure of the Thesis

The thesis started with Chapter 1 containing an introduction of the topic and objectives. Chapter 2 contains information about related theoretical frameworks. Chapter 3 is the literary review section that contains all the relevant information about GenAI tools and software development productivity. 3.1 discusses the utilization of GenAI solutions in software development in general. Chapter 3.2 the GitHub Copilot tool and its usage in improving software development is discussed. The next chapter 3.3 then contains similar facts about ChatGPT. Chapter 3.4 has a comparison of the two tools and their application. Chapter 3.5 discusses the software development productivity and productivity measurements both in general and with generative AI. The next chapter 4 contains the design and results from the case study. The thesis is finished with discussion in chapter 5 and conclusions in chapter 6.

2 Theoretical frameworks

The following sections represent the theoretical frameworks used in the study. The main theories used as the basis of the study are affordance theory and software development theory, as these relate directly to the research questions. The technology adoption frameworks are included to understand the motivation and intention of developers to use a specific tool, and these theories are referred to in the discussion section.

2.1 Affordance Theory

According to affordance theory, most objects gain their meaning through their physical shape. In a physical world, we get perceptions of how to use the objects and features in that world. The perceptions come rather from intuition than carefully thinking about the potential use of the object. This means that the way objects are used rather emerges from the use than a conscious plan for using them. (Matei, 2020)

As opposed to this the use of digital systems emerges from the experience and cultural learning of the person. This means that designers of systems need to think in a very user-centric way. Each on-screen feature has a meaning based on the user's experience. Users have mental maps for organizing the handling of digital objects. They then make their interpretation of the use of the object. That interpretation is also influenced by the context in which the object is used. (Matei, 2020)

2.2 Software Development Productivity Theory

Developer satisfaction and productivity are important aspects of the software engineering process. The satisfaction of developers is related to the possibility of retaining and attracting talent. More productive developers help to improve profits and reduce costs. Many social and technical factors may impact satisfaction and productivity. Although measuring performance through perceived productivity has potential problems in construct validity, so does measuring performance through activity counts. Productivity is not only influenced by various aspects but is also very perceptual. (Storey et al., 2019)

A theory was developed that aims to capture the bi-directional relationships between the two constructs, meaning that more productive developers are more satisfied and satisfied developers are more productive. Social and technical factors relate to perceived productivity and job satisfaction. These factors and their impact may vary depending on context. The theory was derived from related literature and then validated and extended through a survey instrument. The main constructs and relationships are likely transferable to other settings. However, future work is still needed to validate the theory. (Storey et al., 2019) The following figure 1 shows the theory of developer productivity and satisfaction.

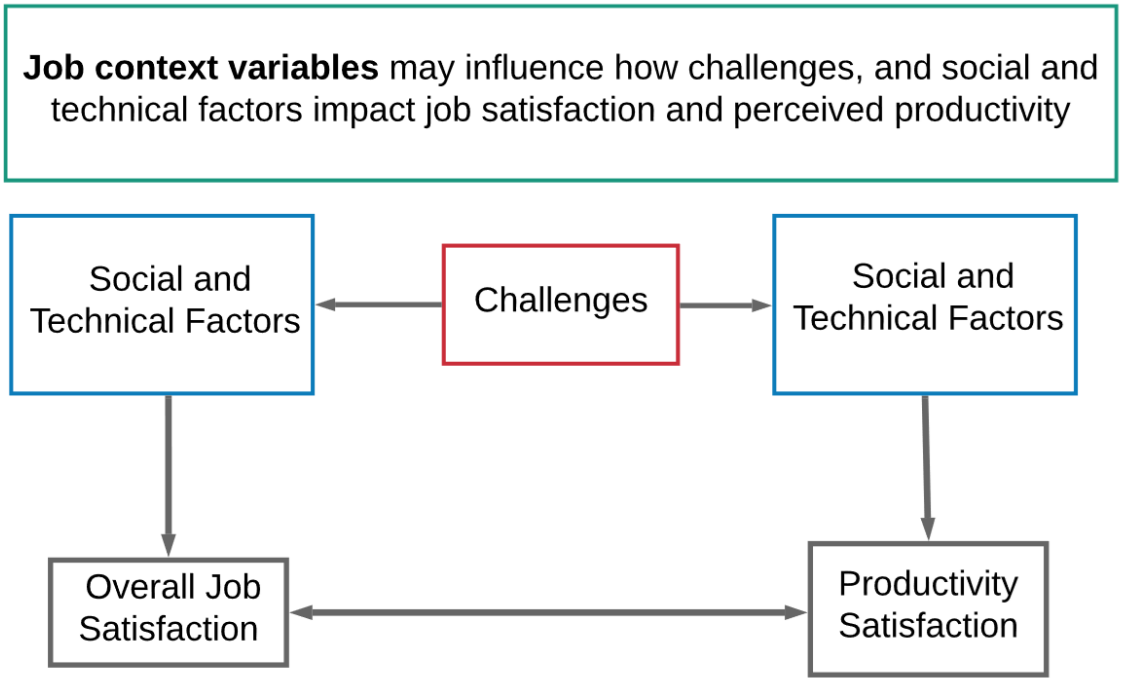

Figure 1. Towards a theory of developer productivity and satisfaction (Storey et al., 2019)

Research shows that many factors impact satisfaction and productivity. These include managers and work culture, impactful work, engineering systems, and non-technical factors. In a case study impact of the manager was found to be the most important factor for job satisfaction. In the same case study 85 % of the developers found that engineering processes and tools were moderately or very important for productivity. (Storey et al., 2019)

2.1 Technology Adoption Frameworks

The technology acceptance model and theory of planned behavior are two theoretical frameworks that can be used as the basis of technology adoption studies in many contexts. The technology acceptance model framework can be used to understand the decision-making factors when taking new technologies into use. The theory of planned behavior is often used in marketing research studies and can also be used to evaluate the acceptance of technologies. Both frameworks are well suited to assessing the adoption of emerging technologies. (Koul et al., 2017)

In a study comparing these two theoretical frameworks in technology acceptance, both were found to explain the intention well. The technology acceptance model explains the attitude towards using a system much better. On the other hand, the theory of planned behavior gives much more detailed information on for example the barriers that people may have in using a system. The models can be effectively used together. The technology acceptance model could for example be used to discover dissatisfied users and the general background for their complaints. Then theory of planned behavior could give more specific information about the group. (Mathieson et al., 1991)

2.1.1 Technology Acceptance Model

Information technologies have the potential to bring many benefits to individuals and organizations. For that reason, the willingness of individuals to accept technologies has been a widely interested researchers. The objective of the technology acceptance model is to reveal the processes that lead to the acceptance of technology. A more practical objective is to give information about the measures needed before implementing systems. Research that was made suggested that an individual decides to perform behavior based on the benefit versus effort they need to put in. The decision to use a system is made be weighting the perceived usefulness of the system and the perceived difficulty of using it. (Marikvan et al.)

Perceived usefulness can be seen as an individual's perception of the way the specific technology improves their performance. Perceived ease of use on the other hand can be seen as the individual perception of the difficulty of using a specific information system. In the model, relationships were defined between perceived usefulness, perceived ease of use, intention and use behavior. (Marikvan et al.)

The technology acceptance model sees the process as having three stages. The designed system triggers cognitive responses, perceived ease of use and perceived usefulness, in an individual: These in turn lead to an affective response which is the attitude toward using a specific information system. The higher the response, the more likely it is that the behavior will take place. The perceived usefulness can lead to direct response which means that it is very important in predicting behavior. Perceived ease of use on the other hand does not

change the behavior directly but affects the overall response. If the solution is easy to use it is more likely to be considered useful for the user. Figure 2 illustrates the technology acceptance model with relations. (Marikvan et al.)

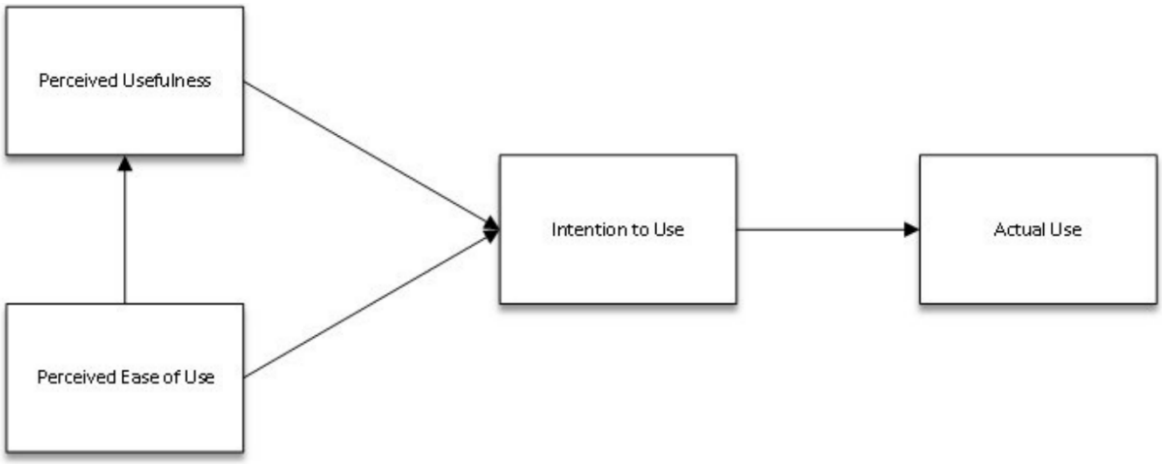

Figure 2. Technology acceptance model (Marikvan et al.)

2.1.2 Theory of Planned Behavior

An important concept in the theory of planned behavior is the intention of the individual to perform behavior. Intentions capture how hard people are willing to try and how much effort they want to put into performing the behavior. With stronger intention, the behavior should be more likely. The intention can only drive the behavior if the action can be taken by will. (Ajxen, 1991)

Behavioral control in important as the resources and opportunities predict the likelihood of achievements. However, perceived behavioral control is a factor that is more interesting as it has an important part in the theory of planned behavior. Behavioral control is something that stays the same for different situations and actions, whereas perceived behavioral control varies between settings. (Ajxen, 1991)

According to the theory of planned behavior, behavioral intention and perceived behavioral control can be used together to predict behavior. With two people having the same amount

of intention for a task such as learning a new topic, the one with more confidence in mastering the subject is more likely to preserve. It depends on the accuracy of perceptions whether the perceived behavioral control can be used as a substitute for actual control. (Ajxen, 1991) In addition to behavioral control, attitude towards the task and subjective norms drive the intention towards the task. Subjective norms can be considered to contain social pressures. This may include the perceived expectations and how highly the individual values those expectations. (Sansom, 2021) Figure 3 displays the theory of planned behavior.

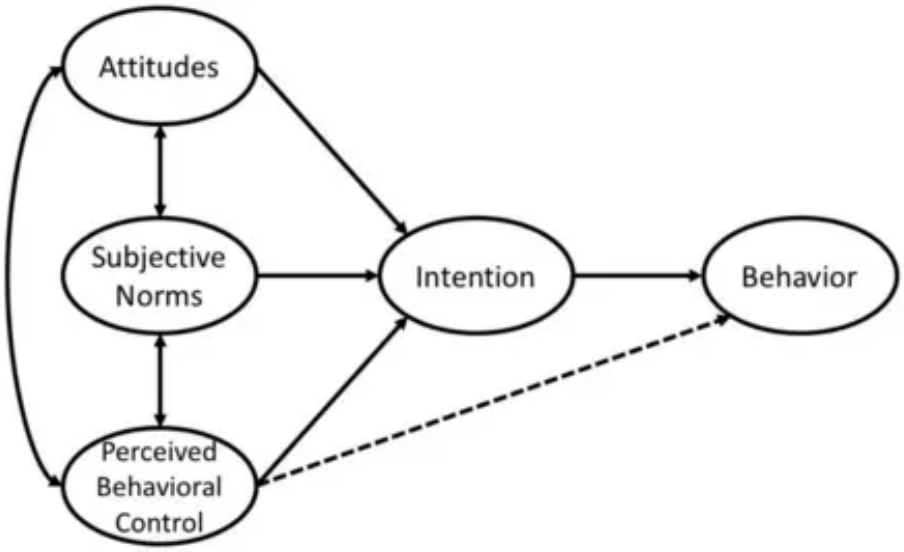

Figure 3. Theory of planned behavior (Sansom, 2021)

3 Literature Review

3.1 GenAI Solutions in Software Development

Over the last few years, GenAI solutions have received considerable attention in software engineering research. Code generation and test case optimization are software engineering tasks that benefit from large language models. (Nguyen-Duc et al., 2023)

In addition to programming, generative AI helps in different areas such as requirements engineering and project management. In programming, generative AI doesn't imply only task automation. This means a significant change in the approach to problem-solving in software development. Software developers move from code-focused roles to collaborating with AI and higher-level architectural decisions. This enables developers to focus on more creative and complex topics. (Nguyen-Duc et al., 2023)

There are many open questions and research areas in GenAI. One challenge is achieving reliable and scalable GenAI output, as the same prompt may produce different answers on different executions. GenAI can also be sensitive to settings and input parameters, and hallucination can be a concern with artificial intelligence-generated content. (Nguyen-Duc et al., 2023)

Based on a recent survey on GitHub for 500-US-based developers, 92 % stated that they already use AI assistants and 70 % found them to improve productivity. (AlOmar et al, 2024) Quite little is known about the usability of LLM-based code generation tools. For instance, how do developers interact with tools that generate good, yet not perfect code? Is it easy for programmers to notice errors in the code? Compared to many other domains, the result, program code, needs to be very close to correct at least to the extent that it compiles. (Vaithilingam et al., 2022)

3.2 GitHub Copilot in Software Development.

GitHub Copilot is an AI pair programmer that is intended to help in efficient code writing. Copilot can offer code suggestions when writing. It can suggest the completion of a current or even a new block of code. The developer can accept all or part of a suggestion or ignore it. The chat feature can be used to ask Copilot how to solve a problem or to explain code. (GitHub, About GitHub)

3.2.1 GitHub Copilot

GitHub Copilot is powered by the GenAI model that has been developed by OpenAI, GitHub and Microsoft. The tool has been trained on public repository code in all available languages. The quality of suggestions depends on the language used as the amount and diversity of training data varies. For instance, JavaScript is one of the best-supported languages as it is well-represented in available repositories. (GitHub, About GitHub)

GitHub Copilot understands the code in IDE through prompts, which are compiled from code and relevant context. Algorithms generate the prompts at any point in coding. The algorithms select relevant code snippets and then prioritize, filter and assemble them into the final prompt. The contextual understanding of GitHub Copilot has continuously matured over time. The first version was only able to consider the file the developer was working on but now GitHub Copilot can process more of the codebase. Neighboring tabs is a technique to process all files in the developer's IDE instead of just one. By including small pieces of context, the suggestions got a better acceptance rate from the developer. (Rosenkilde J, 2024)

GitHub Copilot Chat is a chat interface that makes it possible to have interactions with GitHub Copilot. The chat interfaces make it possible to have access to programming-related information and support without having to search through documentation or online forums. The chat is designed to answer a wide range of coding-related questions such as syntax, programming concepts, and test cases. GitHub Copilot Chat uses both natural

language processing and machine learning to understand the programming question and provide answers. (GitHub, About GitHub Copilot chat)

The language model of GitHub Copilot Chat has been trained based on a large body of text data. The LLM analyses the prompt input given by the user. The chat response can use code formatting features to improve the clarity of the response. (GitHub, About GitHub Copilot, chat in your ide)

3.2.2 GitHub Copilot User Purposes Among Developers

Based on a survey for developers, the purpose for using GitHub Copilot is most often to help generate code. Also, simply curiosity to try out the features is a common reason for use. Often developers turn to Copilot when their code is not functioning as expected and they seek for solutions to fix it. Also, some developers turn to Copilot to improve their ability in programming, for example learning new types of solutions. Another use case is to get new ideas for writing code. (Zhang et al., 2023) Figure 4 below displays the purposes of using GitHub Copilot.

Purpose	**Example**	**Count**	**%**
To help generate code	*GitHub CoPilot was able to fill in the code I needed after I wrote the steps in comments* (SO #71905508)	26	43.3%
To try out the functionality of Copilot	*I use VIM and IntelliJ on a daily basis, and I recently installed VS Code to try out the newest "Copilot Chat" features* (SO #76349751)	9	15.0%
To fix bugs	*spent a month trying to get it to work on my own and after a week of trying to get chatgpt or copilot to fix the bugs I'm all out of ideas* (SO #76266095)	7	11.7%
To improve coding ability	*I'm improving my rusty skills lately and saw (in some Copilot suggestions) the question mark operator used as a prefix of variables* (SO #74008676)	4	6.7%
To provide ideas for writing code	*With the input from jps (AES is actually OK for encrypted tokens, but not signed) and Github Copilot I came up with a working solution using HMAC-SHA256* (SO #72812667)	4	6.7%
For educational purposes	*For the past year I have used and taught my students how they could benefit from co-pilot while coding* (GitHub #19410)	3	5.0%
To generate code comments	*Using ChatGPT and Copilot, I've commented it to understand its functionality.* (SO #75624961)	3	5.0%
To check the code	*I've checked myself by stepping through, I've checked with GitHub Copilot, I've checked with ChatGPT, and they all say this is correct.* (SO #76311798)	2	3.3%
For research purposes	*I am working on a scientific study testing how Copilot will effect the academic setting* (GitHub #8324)	2	3.3%

Figure 4. Purposes of GitHub Copilot use (Zhang et al., 2023)

3.2.3 Challenges with Using GitHub Copilot

One challenge with using Copilot is that developers find it hard to understand the code provided by the solution. One way to make it more understandable would be that the solution would provide explanations of the code. Prior code completion tools only provided one token at a time, but GitHub Copilot can produce longer pieces of code. This also places a demand on the user, as they need to go back and forth from producing code to debugging the code from Copilot. (Vaithilingam et al., 2022) Current research is focused on the code produced by LLMs but the explanations produced by the tools could be just as important. Engineers could easily prefer to accept even a bit more suboptimal solution if it would come with an explanation. (Fan et al., 2023)

Another challenge in using Copilot is the sensitivity of the tool in producing code generated by Copilot. A small difference in a comment can make Copilot generate a very different code snippet. (Vaithilingam et al., 2022) The quality of the code is highly dependent on the conciseness and accuracy of the provided prompt. It could still be seen that GitHub Copilot could be an asset in software development projects when used as a pair programmer. On the other hand, it could turn into a liability when used by those who do not know the problem context or are not familiar with correct programming methods. (Moradi et al., 2024)

As Copilot GitHub is still a new product, the GitHub Copilot's training data will be enriched when more developers start to actively use it. However, usage of the solution may bring troubles if novice developers trust it too much. Copilot could help juniors develop their skills further. (Moradi et al., 2024)

The challenge in using a tool like GitHub Copilot is the way it has been trained. As the solution has been trained on existing source code, simple coding mistakes have been introduced. These can be quite straightforward to fix but other aspects might prove more challenging. These can include introducing program design flaws that cannot be detected and fixed as straightforward. (Pudari et al., 2023)

3.2.4 Productivity improvements with GitHub Copilot

Based on a survey of software developers done by GitHub the developers reported that they complete programming tasks quicker when using GitHub Copilot. More than 90% of developers taking part in the survey felt that the tool helps them to complete tasks faster. The improvement was felt especially in repetitive tasks. In addition to the survey, also a study was done by GitHub to prove the results in a controlled experiment. 95 professional developers were recruited and split into two groups. Another group had the possibility to use GitHub Copilot to complete tasks in writing HTTP server in JavaScript and the other one didn't. The group that used GitHub Copilot had a higher rate of completion (78% compared to 70%) and completed the task 55% faster. (Kalliamvakou, 2022)

In another study, participants were asked to complete a Python programming task using GitHub Copilot, with a control group using a common code completion tool called IntelliSense. Most participants in the study felt that they would want to use Copilot for their programming tasks at work. However, although GitHub Copilot could accurately produce a whole block of code based on prompts, using the tool did not seem to lead to significant measurable productivity improvements. One reason is that although it is easier to produce code through GitHub Copilot than by searching the internet, the code given by Copilot may contain bugs, leading to time spent debugging. (Vaithilingam et al., 2022)

Also, one reason for not seeing the performance improvements in the study is that it may be difficult to understand code that is not produced by yourself. Code searched through the internet is more likely to be correct and comes with discussion and explanations. However, even with the shortcomings, Copilot gives a good starting point for those who may not know how to get forward with problem-solving. (Vaithilingam et al., 2022)

One efficient strategy to finish hard tasks with the help of Copilot is to decompose a complex task into smaller tasks and give prompts for each simpler task for Copilot to solve. This approach can lead to higher efficiency in solving tasks and in the end to a better user experience. (Vaithilingam et al., 2022)

3.3 ChatGPT in Software Development

ChatGPT is a chatbot developed by OpenAI. ChatGPT is artificial intelligence-based and gains its information from many internet-based texts it was trained on. It does not scrape new information from the internet. The public version of ChatGPT was launched to the public in November 2022 and the newer version of ChatGPT 4 in March 2023. A likely scenario is that ChatGPT use will increase productivity among developers, and it is also highly likely that ChatGPT will make software engineering more accessible to the masses. ChatGPT is not only able to take over routine tasks but also non-routine tasks. (Komp-Leukkunen, 2024)

3.3.1 ChatGPT

ChatGPT has already impacted software development practices, as it aids in many tasks such as coding, debugging and testing. ChatGPT can be used in a broad range of tasks. As opposed to Copilot, which is primarily focused on code generation and completion, ChatGPT generates human-like text based on a given input, so-called prompts. (Hao et al., 2024) Chatgpt is trained based on test data it gathers from the internet. Its training process has been based on supervised learning as well as reinforcement learning. OpenAI continuously improves the performance of the solution based on the information it gathers from users. (Yetiştiren et al., 2023)

3.3.2 ChatGPT User Purposes Among Developers

Based on analysis of developers' pull requests, developers seek assistance from ChatGPT across 16 types of inquiries. The conversations can be multi-turn, where the developer can for instance iterate based on previous answers or give a new task for ChatGPT. Developers often do multiple rounds of conversation to improve the quality of responses. (Hao et al., 2024) In another study that was based on the developer’s shared conversations with ChatGPT, it was concluded that the current practice of using ChatGPT is limited to high-level concepts and providing reference solutions. This means that the code generated by ChatGPT rarely ends up as production code. This finding suggests that there is still a lot to

improve in using this type of LLM solution as an integral part of the software engineering process. (Kailun et al., 2024)

The five most common software engineering-related enquiries made by developers are code generation, conceptual enquiries, how-to questions, issue resolving and review. (Hao et al., 2024) The figure 5 below presents the ChatGPT queries classified as PR or issues.

Category	PR	Issue
SE-related:	198(100%)	329(100%)
(C1) Code generation	40 (20%)	90 (27%)
(C2) Conceptual	37 (18%)	45 (14%)
(C3) How-to	26 (13%)	74 (22%)
(C4) Issue resolving	24 (12%)	46 (14%)
(C5) Review	18 (9%)	12 (4%)
(C6) Comprehension	13 (7%)	10 (3%)
(C7) Human language translation	11 (6%)	0 (0%)
(C8) Documentation	10 (5%)	7 (2%)
(C9) Information giving	8 (4%)	8 (2%)
(C10) Data generation	4 (2%)	12 (4%)
(C11) Data formatting	2 (1%)	9 (3%)
(C12) Math problem solving	2 (1%)	9 (3%)
(C13) Verifying capability	2 (1%)	1 (0%)
(C14) Prompt engineering	1 (0%)	0 (0%)
(C15) Execution	0 (0%)	4 (1%)
(C16) Data analysis	0 (0%)	2 (1%)
Others	12	41

Figure 5. ChatGPT queries (Hao et al., 2024)

In the code generation category, developers ask ChatGPT to generate code based on the request. Most often the developers provide the question as text. Also, it is quite common for developers to provide the programming task as code to provide context or adaptation. (Hao et al., 2024)

In conceptual enquiries, developers seek clarification for principles, practices, tools or general implementation ideas. This means that ChatGPT is not used by programmers only as a programming assistant but also to understand technical knowledge they are not

familiar with. Often developers also turn to ChatGPT with initial how-to questions, when they are in the beginning phases of problem-solving and want step-by-step instructions. (Hao et al., 2024)

Often ChatGPT is also used for issue resolving. This can mean for example sharing a code-related error message or asking for help in non-code related tasks such as setting up a system. Occasionally developers also provide ChatGPT external links, expecting the tool to be able to access the link and analyze the problem. (Hao et al., 2024) Also requests to review code are common as developers request to compare different implementation options or to offer suggestions for code improvement. Developers tend to seek external validation for implementation choices as they make pull requests. (Hao et al., 2024)

As LLMs are nondeterministic by nature, this aspect needs to be considered for ChatGPT in code generation. The code generated by ChatGPT varies a lot on the same given prompt. This inconsistency implies that there may be errors in the resolutions. Interestingly giving more detailed and lengthy instructions results in more variation in the output and potential errors. (Ouyang et al., 2023)

In most cases, the code supplied by ChatGPT is merely supplementary information for the developer. This may be due to unexpectedly low quality. Sometimes the code from ChatGPT may be used as a document, for instance, to generate a readme file. In many cases, the developer modifies the code a lot before merging it into the primary project. Sometimes the code ends up being used as it is directly. (Kailun et al., 2024) The figure 6 below represents the distribution of amounts in follow-up questions from ChatGPT.

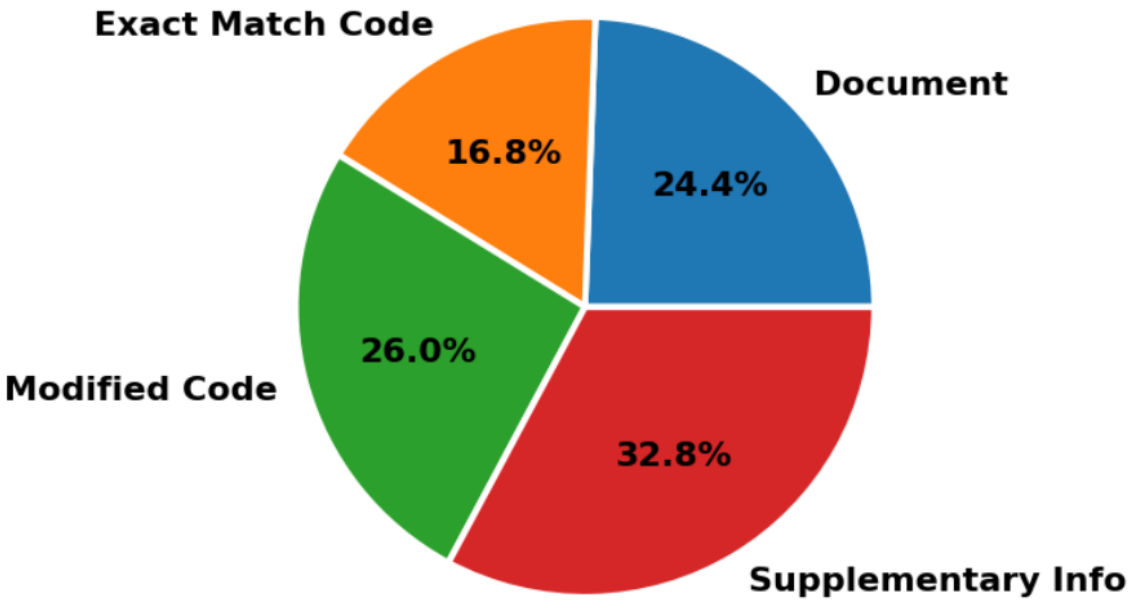

Figure 6. The distribution of follow-up questions given for ChatGPT (Kailun et al., 2024)

One use case for ChatGPT is also that it can help individuals learn software engineering, for example at university. It depends on how ChatGPT is adopted and whether they will acquire relevant knowledge. It is also possible that individuals without previous knowledge may be able to gain the needed understanding by themselves. This might mean that the demand for software engineers would decrease. (Komp-Leukkunen, 2024)

3.3.3 Interaction With ChatGPT

Interaction with GenAI solutions is typically done through prompts, which means giving instructions and context information for ChatGPT. A way to use a prompt can be to directly ask the solution to provide code. (White et al., 2023) Prompts are a form of programming to customize the output from LLM. The quality of the output from conversational LLM is closely related to the quality of prompts. (White et al., 2023)

Prompt patterns serve the need to create reusable patterns in interaction with conversational agents. The prompt patterns can be seen as very similar to other types of software patterns, with the difference that prompt patterns focus on optimizing the output of conversational LLMs. (White et al., 2023) First jobs for separate prompt engineers have

already emerged, some researchers suspect those might disappear once developers gain their skills in prompt engineering. (Komp-Leukkunen., 2024)

When developers make multi-turn conversations with ChatGPT, they interact with the conversational agent in multiple ways. These include the three most common types: those that contain follow-up questions, ones that contain the actual initial question, and those that are refined from the previous prompt. (Hao et al., 2024)

When making a follow-up question, it is typically to improve the first solution provided by ChatGPT. This can be to repair a programming code that ChatGPT has suggested in the first round as a response to a complex programming problem. In the second most common type of follow-up, the developer gives the actual question in the second prompt, so the first prompt may be to give background information. In the third most common type the developer posts the same question as in the initial prompt but with additional information. The attempt in this type of prompting is to improve the response quality. Other follow-up questions include information giving, revealing a new task, giving negative feedback and asking for clarification. (Hao et al., 2024) The figure 7 below describes the common flow in multi-turn conversations.

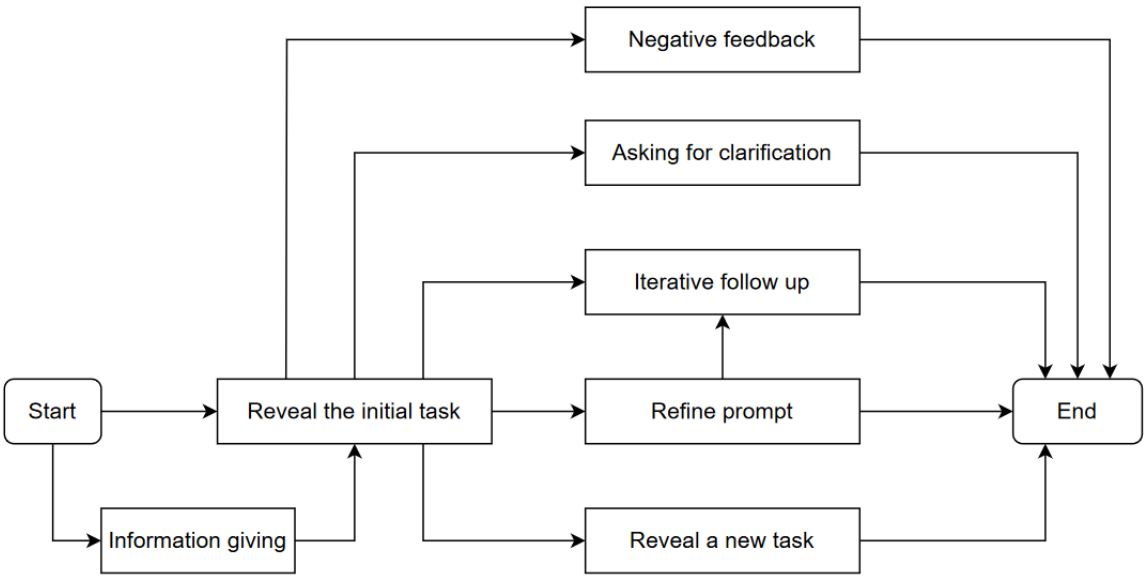

Figure 7. The flow of follow-up questions given for ChatGPT (Hao et al., 2024)

In a study, the participants felt that conversating with ChatGPT was fun and increased engagement. Interacting with ChatGPT was faster and more natural engagement than searching for topics on Google search. Also, the threshold of asking from the conversational agent was lower than asking another person. From GenAI, you get the response directly while when asking from a colleague you might need to wait for a while. (Ulfnes et al., 2024)

3.3.4 Prompt Patterns

Prompt patterns make it possible for the developers to make fast experimentation with different approaches throughout the whole software development process. The depth of LLM capabilities that solutions like ChatGPT offer is still not quite fully appreciated. The solution can be used at different abstraction levels. Different prompt patterns can be integrated into pattern catalogues to make them effective. Using the patterns has the potential to accelerate development and make exploring different options faster. (White et al., 2024)

An example of a pattern would be an ‘architectural possibilities’ pattern. Using the pattern generates multiple different architectures for consideration. These options can consist for example of different alternatives of how communication is done between modules or tiers. This pattern would be beneficial as considering different architectural possibilities against requirements is a significant effort from the development team. Also, the developers may not be familiar with all the related architectural options. The pattern of asking for architectural options could contain an explanation of what the developer intends to do in the project, descriptions of constraints and requests to describe possible architectures for the system. (White et al., 2024)

Another example of a pattern is a principled code pattern. The goal would be to ensure that code follows a named coding principle without needing to describe each design rule. Following principles ensures that the code developed is maintainable. The prompt pattern would be first to state the scope and then ask the GenAI tool to generate, refactor, or create code to adhere to principle x. (White et al., 2024)

3.3.5 Challenges With Using ChatGPT

The weakness of ChatGPT is that it has a limited understanding of the context of the codebase. Because it does not understand the codebase at hand, it makes it difficult for it to understand the dependencies of the code. ChatGPT may misunderstand the given code excerpt, especially in those cases where the relevant code is too large to provide. (AlOmar et al., 2024)

Also, the quality of ChatGPT responses depends a lot on the quality of the prompts. This also means that there is a need to promote awareness among developers of the importance of prompt engineering. Developers sometimes rely too heavily on the model's ability to understand the questions without giving a proper context. ChatGPT often also performs better when it is given information about the intent or how the task should be performed. (AlOmar et al., 2024)

3.3.6 Productivity Improvements with ChatGPT

In a study, a software project was conducted to complete a software solution for financial supervision with the help of ChatGPT. The goal was to understand the effectiveness of the tool particularly in a more holistic role instead of just creating code snippets. The study concluded that there were clear benefits, but also challenges in the workflow. The AI assistant reduced the time required to make the initial project setup and to create the foundation of the code. However, in more complex cases more intervention was needed, for instance with complex SQL queries and database schema design. From the perspective of producing a production-ready solution with the help of ChatGPT particular attention needs to be given to prompts, as these significantly influence the effectiveness of the tool. Also, rigorous quality assurance needs to be done so that for example validation logic is not missing from the code. Experienced developers may be needed to provide efficient prompts and validate the code. (Liukko et al., 2024)

In a study aiming to identify possible future scenarios of ChatGPT, diverse experts were interviewed, and five main scenarios were identified. These scenarios are 1. Status quo is

maintained 2. There are advancements in learning and teaching thanks to ChatGPT 3. ChatGPT is integrated into startups 4. The productivity of software engineers is increased. 5 ChatGPT takes over the jobs of software developers. The likelihood of the scenarios and how far in the future those would most likely happen is described in figure 8 below. (Komp-Leukkunen, 2024)

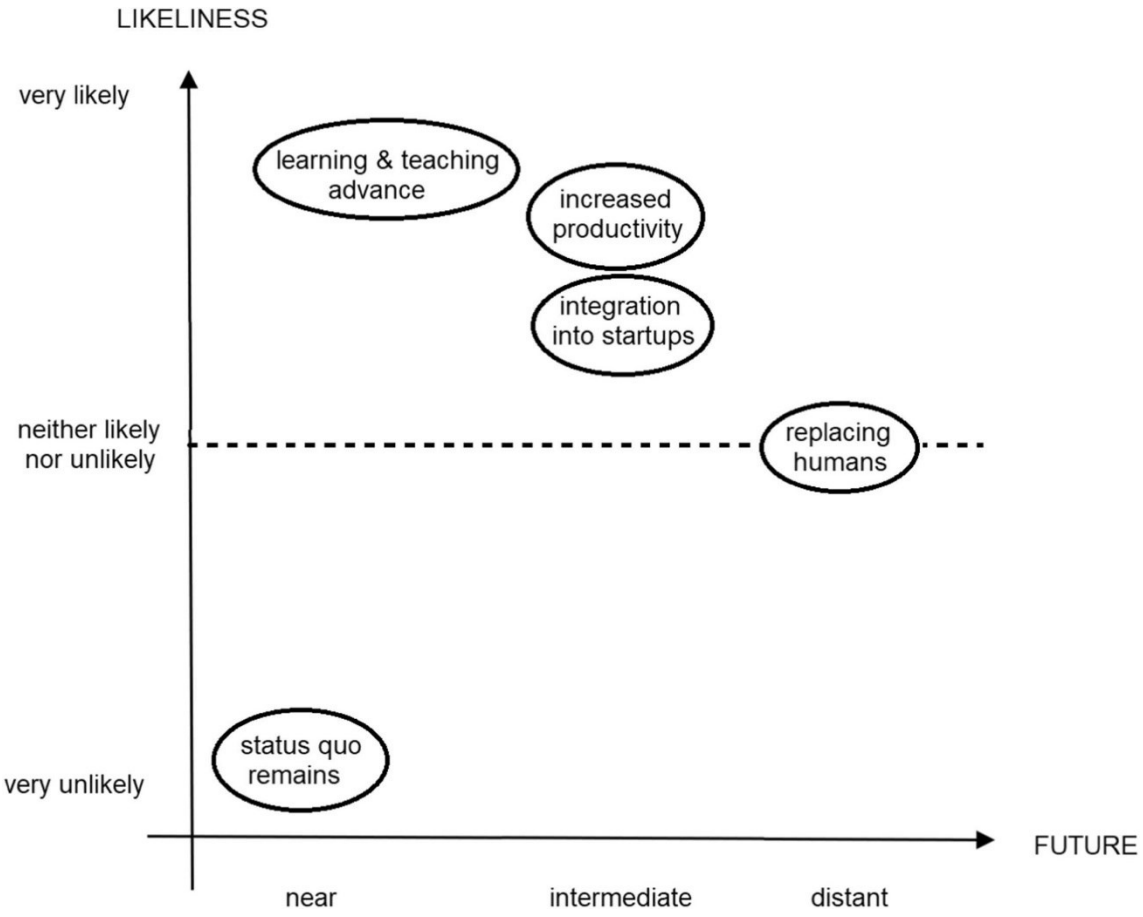

Figure 8. Future scenarios of using ChatGPT. (Komp-Leukkunen, 2024)

When considering the possibility of increased productivity in software when using ChatGPT, the respondents see that in the intermediate future, it is likely that people get skilled at using ChatGPT and use it in their daily work. In this scenario companies expect developers to see ChatGPT increase productivity. The respondents still considered that ChatGPT should be developed further to become reliable enough to use in established companies. So, this scenario requires an improved version of ChatGPT to be released. The

respondents saw this improvement and increase in productivity as likely to happen in the intermediate future. (Komp-Leukkunen, 2024)

The respondents in the study also considered how likely the consequences of the productivity scenario were. The consequences considered were that “Some jobs for software developers will disappear”, “Software developers must use ChatGPT” and “There is employment only for very skilled developers”. The respondents thought that the two first responses were between likely to very likely to happen soon. The last consequence of only having jobs for very skilled developers was estimated on average somewhere between likely and unlikely. (Komp-Leukkunen, 2024)

3.4 Key Differences Between GitHub Copilot and ChatGPT

As ChatGPT has been trained as a more general-purpose tool, it can answer to wider range of questions than GitHub Copilot. GitHub Copilot is more directly related to code completion (Lawton. 2023). In a study among developers within media technology organizations, both Copilot GitHub and ChatGPT were used among developers to perform pair programming tasks. According to developers, the tools benefitted them in different ways. GitHub Copilot was a smart autocomplete, as it analysed the code and offered suggestions to speed up development. Cooperation with ChatGPT was more like discussions with other developers as they sought assistance and suggestions. Developers could engage in conversations with ChatGPT, clarify ideas and then implement those with the help of GitHub Copilot. (Pereira et al., 2024)

In the same media technology organization study, it was found that the performance of GitHub Copilot was highly dependent on technology and applications. Copilot had trouble understanding the older conventions, programming practices and technologies. Developers found Copilot to be most helpful with new projects. When a solution has been developed using modern and good practices, GitHub Copilot works well in understanding the context within the code base. ChatGPT does not have similar limitations to legacy code, as it provides good answers even with outdated technologies. (Pereira et al., 2024)

One of the main differences between GitHub Copilot and ChatGPT is that ChatGPT does not have IDE support. (Yetiştiren etl al., 2023) The lack of tool integration means that there is a lot of copy-pasting of code and text between windows. Having the tool available in a seamless process would reduce the work a lot. (Ulfnes et al., 2024) ChatGPT does not have specific supported programming languages while GitHub Copilot has a list like this. (Yetiştiren et al., 2023)

GitHub Copilot can provide multiple recommendations for a problem, but ChatGPT provides one suggestion at a time unless specifically asked to provide multiple options. ChatGPT is useful also for other purposes than directly programming related questions and it explains its suggestions in detail. GitHub Copilot can access user's local files while ChatGPT cannot access them. (Yetiştiren et al., 2023)

ChatGPT is trained based on human language data while GitHub Copilot is trained on a large amount of source code. (Lawton. 2023) The amount and quality of pre-training data relate widely to the accuracy of responses from a language model. ChatGPT uses training data from various text sources. Copilot is trained on curated and licensed datasets from diverse sources. The size of the language model makes a big difference in the performance of the solution. (Mandvikar, 2023) Because of the increasing complexity and size of codebases and several new technologies, GenAI models may need to include more and more dimensions. The race is getting more intense between the large technology corporations. (Nguyen-Duc et al., 2023)

A study assessed publicly available LLMs such as GitHub Copilot Chat, ChatGPT, Google Bard and Microsoft Copilot. The tasks included code documentation and explanation, solving student assignments and learning new concepts. When it comes to code explanations Microsoft Copilot provides detailed comments and explanations for all programming languages. The tool also provided logical reasoning. ChatGPT performed similarly to Copilot but was unable to give logical reasoning in some of the prompts. GitHub Copilot Chat showed good logical reasoning abilities but was unable to provide in-depth explanations, especially when required to give longer explanations in plain English. (Agarwas et al., 2024)

In programming assignments, GitHub Copilot Chat was the strongest performer. It understood the assignment and answered reliably. Bard had responses that were of moderate quality. ChatGPT had a good understanding of the tasks, and it provided a good overview of the solution, but the responses were lacking in some technology-specific details. Microsoft Copilot was the worst performer in this task, as its responses were vague and lacked depth. (Agarwas et al., 2024)

When it comes to using LLMs to understand new concepts and frameworks, ChatGPT excels in explaining concepts in plain English. GitHub Copilot Chat is unable to give explanations in clear English in non-coding tasks. Microsoft Copilot was found to be lacking in clarity and detail, although it could provide relevant information. Google's Bard had the best performance in the task as it gave clear responses consistently with strong logic. (Agarwas et al., 2024

DAnTe is a defined Degree of Automation Taxonomy for software engineering. It has six levels (Level 0 to Level 6), where Level 0 means that no automation is employed, and Level 5 means complete automation of the process. (Melegati, 2023) These levels are displayed in figure 9 below.

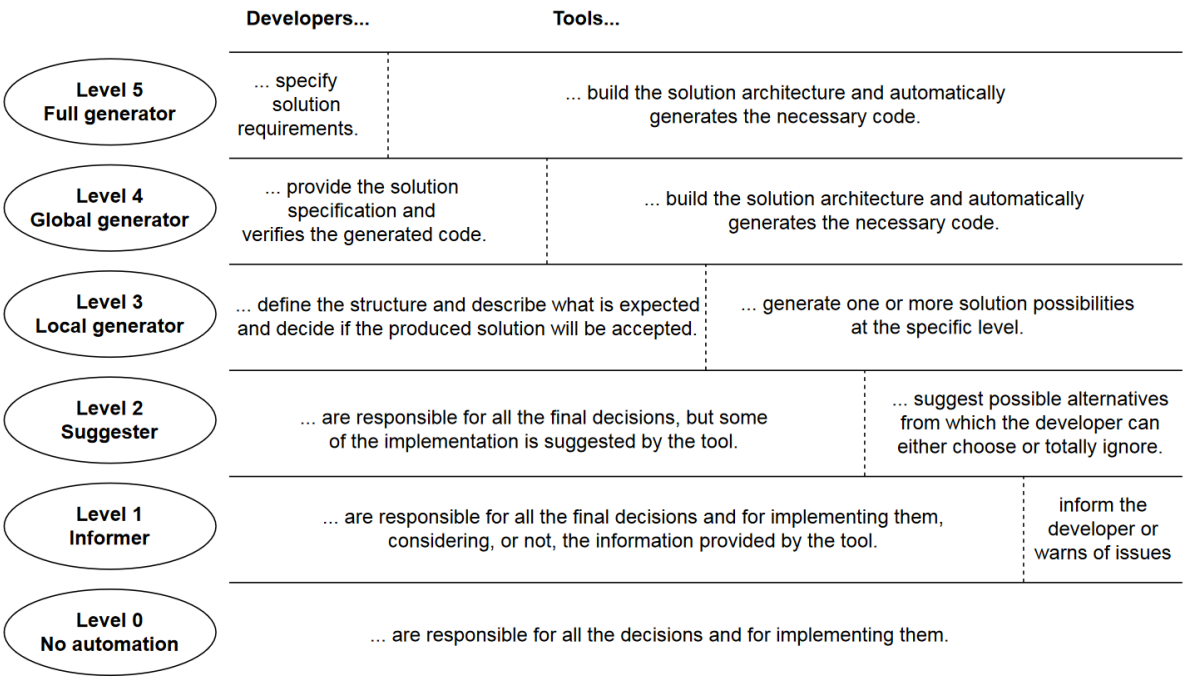

Figure 9. DAnTe levels of software engineering automation. (Melegati, 2023)

An example of a Level 1 informer would be a simple IDE that checks the code and warns if problems are found but does not propose a fix. On level 2 an example would be a tool that suggests modifications to code, such as simple auto-completion tools. The developer then decides whether to incorporate the changes or not. Level 3 local generator tools automatically perform tasks on a limited scope based on the brief description given by the software engineer. GitHub Copilot can perform at this level as it can compose methods or even classes based on comments. However, the overall conception and structure of the code is still defined by the developer. At Level 4 of a global operator, the LLM should be able to provide complete solutions based on instructions. A developer is needed to evaluate and sometimes modify the system to meet the needs. Some studies have evaluated ChatGPT for this purpose, but the results have still been modest. Most tools capable of working at this level are more limited, for example without actual code production. At Level 5 of the full generator, the tool would be able to build the entire solution without changes needed from the developer. (Melegati., 2023)

When examining lessons learned from using the tools, it appears that ChatGPT and GitHub Copilot complement each other rather than act as alternatives. They often support different

use cases and even when they support the same use cases they do so in different ways. (Pereira et al., 2024)

Two main modes of using artificial intelligence assistants have emerged, exploration mode and acceleration mode. In acceleration mode, the developer already knows the type of code they are trying to produce and merely uses the tool to complete the code more quickly. In exploration mode, the developer is more unsure about the solution and would like to explore options. (Liang et al., 2023)

In a survey study made for many developers, it was found that developers seek assistance from AI programming assistants in both modes, while acceleration mode may be preferred. The developers seem to be most motivated to use AI programming assistants in tasks such as autocompletion rather than in brainstorming solutions for problems they are facing. While GitHub Copilot was the most popular tool in the study, many people like to use ChatGPT because of its ability for natural language interactions. Although this type of interaction is promising, there is a question of when developers should rely on these interactions. Some users might also prefer using chat-based interactions in acceleration mode. However, this is a usability issue and the user's cognitive load needs to be considered. Clear explanations could also provide value as is often the best way to receive good output from using the tools (Liang et al., 2023).

3.5 Improving and Measuring Software Development Productivity

To consider the potential for productivity improvements with GenAI, software development productivity needs to be understood. The following sections present software development productivity and related productivity measurements, and then generative AI productivity improvements and related measurements.

3.5.1 Software Development Productivity

Development is a process with complicated interactions between different stages. This means that the stages cannot be considered in isolation but rather as a whole. Also,

an understanding of productivity must be based on an understanding of how developers spend their working time. Based on various studies, software developers spend surprisingly little time writing code, from 9% to 61%. They for example collect information from meetings, read documentation, search through the internet, help co-workers, and perform administrative duties. (Meyer et al., 2018)

There is a vast amount of research showing that factors such as frequent interruptions lead to lower task performance. What is less studied are the human aspects such as job satisfaction and its impact on productivity. (Meyer et al., 2018) The satisfaction of developers and work productivity are related. For this reason, those need to be considered carefully in software companies. Being more productive may lead to developers being more satisfied and more satisfied developers may be more productive. Introducing new technology such as GenAI can improve work satisfaction (Ulfnes et al., 2024) Job satisfaction is greatly affected by the amount of good and bad workdays that define how happy the developer is about their immediate work situation. (Meyer et al., 2018)

Developers consider a workday good when they can progress and create value on the projects. Many developers are dissatisfied when they work on tasks that they consider mundane or repetitive. To make good days typical it is good to avoid interruptions such as unnecessary meetings or administrative tasks in the active development phase. (Meyer et al., 2018)

3.5.2 Measuring Software Development Productivity

SPACE framework aims to measure the most important aspects of developer productivity. Measuring productivity with more than a single metric helps to better understand how people and teams work so that better decisions can be made for improvement. (Forsgren et al.,2021)

One aspect measured in SPACE is satisfaction and well-being. This is beneficial to measure since productivity and satisfaction are correlated and a decline in satisfaction could signal upcoming reduced productivity. What could for example be measured is employee satisfaction and developer efficacy, meaning whether developers have the

necessary tools and resources to get their work done. The second aspect to measure is performance, which is the outcome of a process or system. This may be hard to quantify but could be measured through for example quality or impact, for example, customer satisfaction or feature usage. (Forsgren et al., 2021)

The third SPACE aspect is the count of actions or outputs produced. One type of developer activity that can be measured includes coding metrics such as the number of items or commits. The fourth category, collaboration and communication metrics can be used to categorize how teams communicate and work together. The fifth and last category, efficiency and flow, aims to capture the ability to make progress or complete work without too many disruptions or delays. This could be measured for example by the number of handoffs or number of interruptions. (Forsgren et al., 2021) An example model of Spice metrics is included in figure 10.

FIGURE 1: **EXAMPLE METRICS**

LEVEL	SATISFACTION & WELL-BEING How fulfilled, happy, and healthy one is	PERFORMANCE An outcome of a process	ACTIVITY The count of actions or outputs	COMMUNICATION & COLLABORATION How people talk and work together	EFFICIENCY & FLOW Doing work with minimal delays or interruptions
INDIVIDUAL One person	*Developer satisfaction *Retention† *Satisfaction with code reviews assigned *Perception of code reviews	*Code review velocity	*Number of code reviews completed *Coding time *# Commits *Lines of code†	*Code review score (quality or thoughtfulness) *PR merge times *Quality of meetings† *Knowledge sharing, discoverability (quality of documentation)	*Code review timing *Productivity perception *Lack of interruptions
TEAM OR GROUP People that work together	*Developer satisfaction *Retention†	*Code review velocity *Story points shipped†	*# Story points completed†	*PR merge times *Quality of meetings† *Knowledge sharing or discoverability (quality of documentation)	*Code review timing *Handoffs
SYSTEM End-to-end work through a system (like a development pipeline)	*Satisfaction with engineering system (e.g., CI/CD pipeline)	*Code review velocity *Code review (acceptance rate) *Customer satisfaction *Reliability (uptime)	*Frequency of deployments	*Knowledge sharing, discoverability (quality of documentation)	*Code review timing *Velocity/flow through the system

† *Use these metrics with (even more) caution — they can proxy more things.*

Figure 10. Developer productivity Spice mode. (Forsgren et al., 2021)

3.5.3 Productivity Improvements Through GenAI Solutions

McKinsey Digital's empirical research found significant improvement in using GenAI for many development tasks. The research results include both tools connected to IDE and conversational agents. The results indicate that code can be documented for maintainability in half the time, new code can be written in nearly half the time, and code refactoring in

almost two-thirds the time. These speed improvements can be translated to increased productivity in engineering. However, the time savings vary depending on the complexity of the task and the amount of experience the developer has. The time savings were only less than 10 percent in tasks perceived as high complexity. Also, developers with less than one year of experience had similar 10 percent improvement levels. (McKinsey Digital, 2023) Figure 11 below displays the impact of Generative AI on development speed.

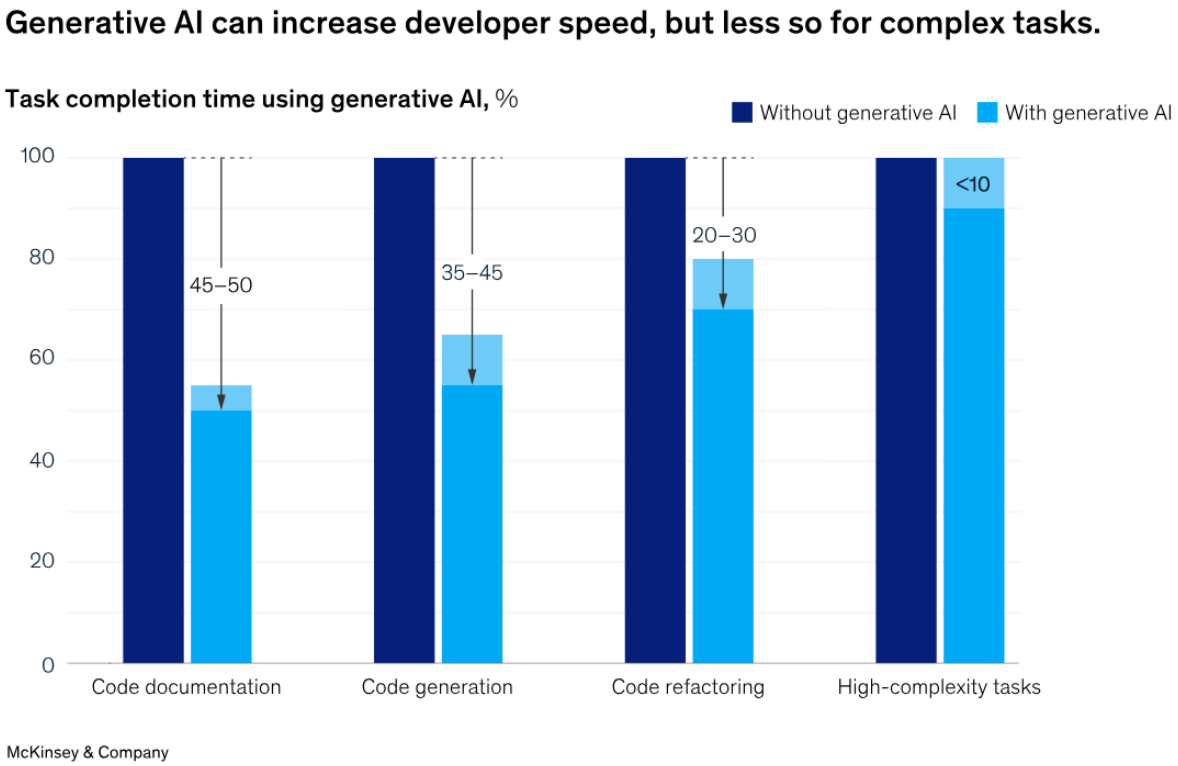

Figure 11 Generative AI impact on developer speed (McKinsey Digital, 2023)

Software systems need more complex work done than simple coding work. AI assistants are on their way toward useful support, but more abstract challenges are currently not solvable for them. Solutions need to be modified to address the needs of many stakeholders and aspects such as performance and security must be considered. Automating the design process is challenging as it is the most abstract part of software development. AI-assisted software development where the AI supports developers in more complex tasks is likely to be possible in the future. (Pudari et al., 2023)

When taking Gen AI tools into use there can be a significant change in the dynamics of communication. Some developers appear to consult AI solutions for issues that they would

have previously discussed with human colleagues. This may have a significant impact on the team's performance. The team’s performance depends highly on the knowledge sharing between the team. For this reason, reducing knowledge sharing by including generative AI should be observed. Problems become more personal, and people may get less support from others. So, while it can be expected that Generative AI has a positive impact on individuals’ performance, there is a risk of reducing overall team performance. (Ulfnes et al., 2024) Furthermore, implementing new ways of working can be challenging for people and organizations. It can require a lot of time and training to bring new tools into use. For that reason, companies need to be clear about the benefits before taking generative AI tools into use. (Pereira et al., 2024)

Even though there is potential for productivity improvements with the tools, there are factors that should be considered as possible limitations. The tools only deliver productivity improvements if the developer can provide value with the saved time. On average the developers spend only part of the day in direct coding activities, resulting in two to three hours per day. If the GenAI tool halves the time spent on coding, there would be about an hour saved. This time should be spent effectively to have actual productivity improvements. (Gartner, 2023)

Another point to consider is how maintainable the code provided by the solutions is. For the productivity benefits to be sustainable, the developers must understand the generated code. Otherwise, they will not be able to maintain it. It is also possible that by more careful inspection of the code, more sophisticated errors could be found. (Gartner, 2023)
Three aspects are directly related to the SPACE framework where the focus should be. These are 1. satisfaction and well-being 2. communication and collaboration and 3 efficiency and flow. (Gartner, 2023) Figure 12 below shows these focus areas.

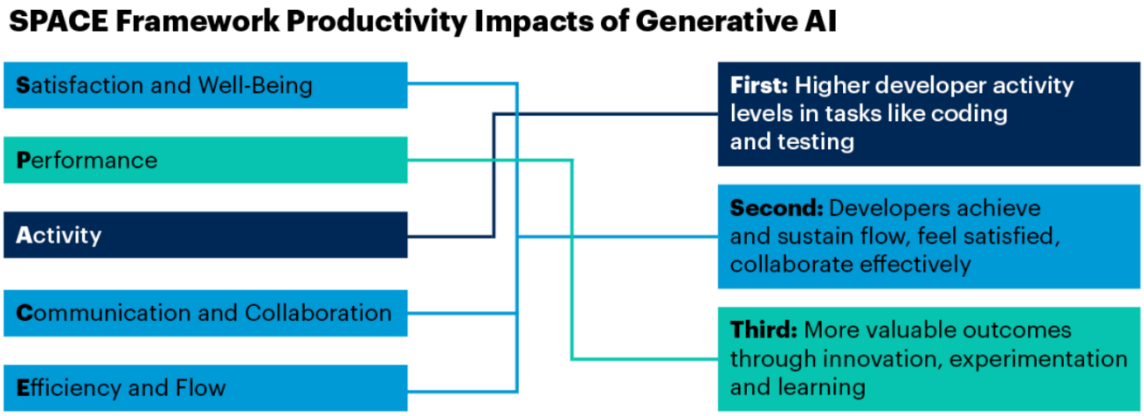

Figure 12. Generative AI focus areas in SPACE framework (Gartner, 2023)

The potential for performance improvements using code completion tools is huge, but measuring the impact on productivity is challenging. Measuring developer productivity based on activity counts undermines the complexity of software development. (Ziegler et al., 2022) The higher number of activities can be for example an indication of bad planning, as the developer must overwork to meet the deadline. Or it could for example indicate that the overall engineering system has gotten better resulting in developer being able to perform their jobs more effectively. Measuring activity counts alone would not tell the reason behind the improvement. (Forsgren et al., 2021) Measuring the self-perceived productivity of developers along with automatically measured data gives a much more complete picture of productivity. (Ziegler et al., 2022)

When measuring the productivity improvements of using GitHub Copilot, the acceptance rate of suggestions is a better predictor of perceived productivity than other metrics. Acceptance rate can be used as a rough monitoring of the performance of neural code synthesis systems such as GitHub Copilot. There is a significant difference in acceptance rates between different languages. For instance, typescript has an acceptance rate of around 24,7% and Python 14,1%. (Ziegler et al., 2022)

Simply focusing narrowly on the correctness of suggestions might not tell the complete story when it comes to the usefulness of Generative AI assistants. A code suggestion inside an IDE can be seen almost like a conversation with a chatbot. A suggestion can serve as a useful starting point for tinkering on complex programming tasks. This can lead to more value than providing a suggestion that only serves to save time in typing. (Ziegler et al.,

2022) The acceptance rate of Copilot's suggestions is an important factor in perceived productivity. However, sometimes developers simply accept the suggestions when they are displayed and focus on evaluating their feasibility later. (Hao et al., 2024)

Furthermore, all the benefits and features of having a pair programmer cannot be measured by quantitative metrics. Pair programming with a Generative AI assistant is a human-tool interaction and understanding the developer's experience when using such tools needs to be also considered for a comprehensive view of the subject. (Moradi et al., 2024)

There are also other factors to consider when focusing on the acceptance rate. For instance, focusing on improving the acceptance rate has a bias toward using the most popular programming languages and IDEs. Also, the acceptance rate could be artificially improved for example by asking for coding suggestions in smaller pieces at a time. (Ziegler et al., 2022)

Based on a survey by GitHub for 2000 GitHub Copilot users, the assistant seems to significantly improve developer satisfaction. The survey contained statements that covered all dimensions SPACE framework. Space is a framework that includes several aspects of productivity. These include 1. Satisfaction and well-being 2. Performance 3. Activity 4. Communication and collaboration 5. Efficiency and flow. Below is figure 13 focusing on the survey results of the aspects and satisfaction and well-being, efficiency and flow. (Kalliamvakou, 2022)

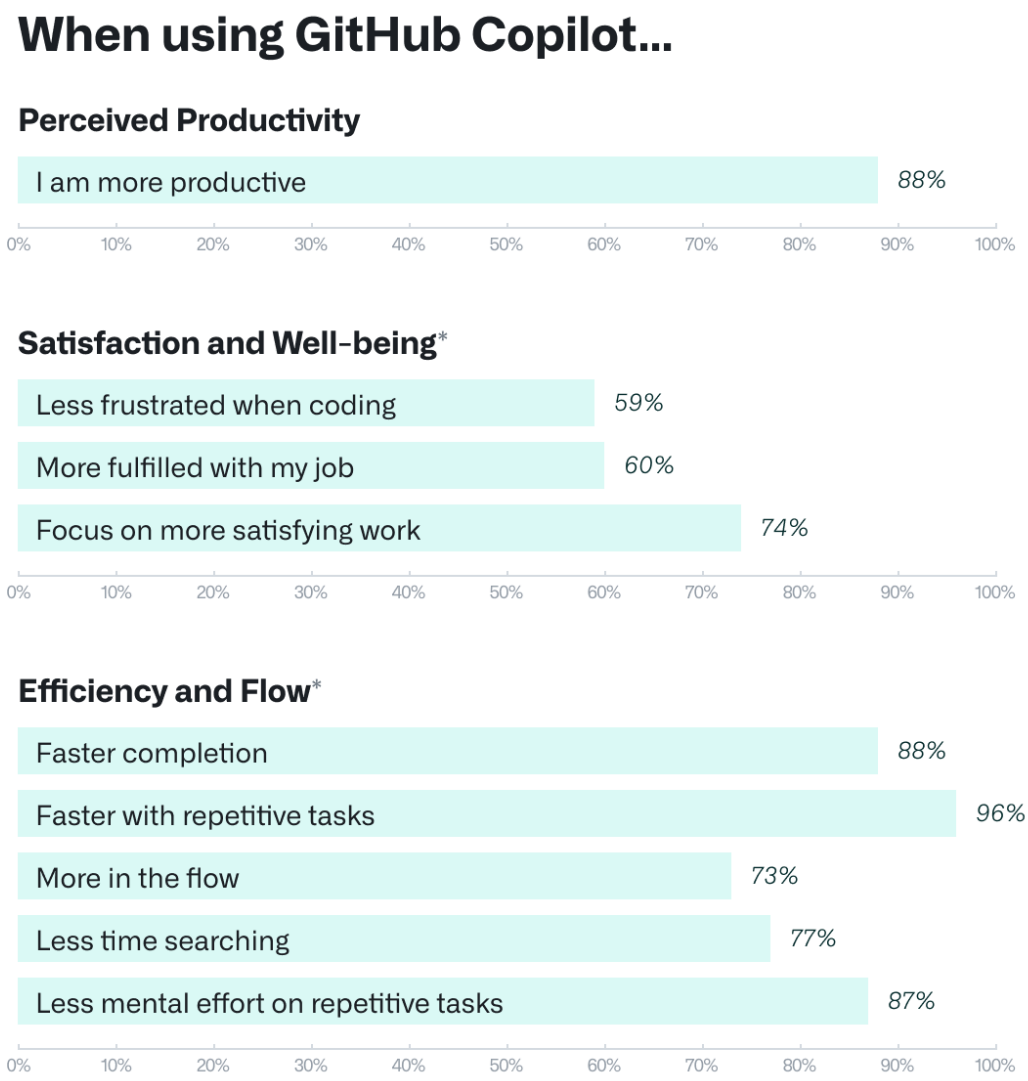

Figure 13. GitHub copilot survey results. (Kalliamvakou, 2022)

4 Case Study of GenAI Adoption in Software Development

To deepen understanding of the performance improvement benefits of GenAI tools, information was gathered in a selected company. The following sections contain the case study design, data collection and analysis.

4.1 Case Study Design

When designing a case study several areas should be considered. These include the objectives of the case study, methods of data collection and analysis. (Runeson et al., 2012) The literature review section gave a good background for refining the objectives of the case study. The company selected for the case study is a large Finnish company employing around 40 developers in digital product development. The company is likely to be more advanced in using modern tools and approaches in programming. For this reason, there was interest in the study as the personnel seeks to actively benefit from generative AI tools.

One objective of the case study is to understand how developers and others experience the productivity benefits from the tools. Also, it would be interesting to evaluate if developers gain similar productivity improvements from both types of tools. According to the software development productivity theory by Storey et al. (2019), productivity and satisfaction are closely linked to each other and have a bi-directional impact. Also, the engineering system, that the GenAI tool is part of, may have an impact on both productivity and satisfaction. The hypothesis is that generative AI solutions improve productivity and work experience from the developer's point of view. Another objective is to understand what use cases are seen as most beneficial for the usage of generative AI tools. The separate tools support different use cases in different ways. Also, according to the affordance model people see different opportunities in using the tools based on their background (Matei, 2020). It is beneficial to see what possibilities have been found in the case company. If possible, the intention is also to provide some actionable insights on how the organization can proceed in the journey of adopting generative AI tooling.

Interviews are one of the most commonly used sources of data in software engineering case studies. The reason for using this format is that a lot of knowledge is only available in the minds of people working around the case. The interviews can be categorized into three types: unstructured, semi-structured, and fully structured. In unstructured interview, the interview questions are general interest areas of the interviewer. (Runeson et al., 2012) The interviews in this case study were unstructured, as only themes for the discussion were planned. The reason for this was that the interviewed people had a lot of information on the subject and discussed it openly through presentations. Also, the role of the discussions was

to serve the more structured data collection method in this case study, the survey. The data collected from the interviews is presented in the following chapter discussing the background information of the case company.

A questionnaire-based survey is quite like a fully structured interview as all the questions are planned and detailed (Runeson et al., 2012). To gather the relevant information from a big enough sample of developers, a survey was done in this case study. The survey design for data collection is presented in section 4.3 and the results in section 4.4. The data collected in the case study is further analysed in the discussion section along with the information gained from the literature review.

4.2 Background Information of Case Study Company

The company where the case study was done is a large Finnish company employing around 40 developers in digital product development. This digital product development organization is at focus when discussing the GenAI tooling adoption in their software development. The teams use various programming languages in their daily work. The adoption of GenAI tooling was started 1,5 years ago. At the time knowledge about the potential and features of the tool was still light. Currently, the skill level and amount of information on the technical features of the product is much higher.

When starting the GenAI adoption there was a prompt engineering workshop and call to action to especially start using GitHub Copilot. According to the engineering manager, all 40 developers have taken GitHub Copilot into use in their daily work. The experiences have been positive in improving the programming productivity. Some individual developers may use ChatGPT, but it has not been taken into wider use.

Within the last half year, the management has started to get numbers out from the GitHub Copilot usage. The numbers are related to the acceptance rate, for instance, the average acceptance rate and ratio of suggestions to acceptance rate. Also, the acceptance rate by a programming language is shown. The acceptance rate has not been giving much information to management about the actual productivity improvements. By itself, the number does not tell much as it is hard to interpret the actual productivity impact and why

the number varies. From the GitHub Copilot acceptance rates, it can also be seen that the rates vary by programming language.

In addition to acceptance rates, many other factors related to productivity are measured on the software engineering scorecard. Measurements include factors such as cycle time, throughput, and review rate. Since starting to use GenAI tools there has been significant positive change in the numbers. However, at the same time, other improvements have also been made. For that reason, it is difficult to attribute exactly how much of the improvement can be allocated to taking GenAI tools into use. It has been thought that there could be two groups of developers for some time, one that uses GitHub Copilot and the other that doesn't. When asked after participants for such a study there have been no volunteers to give up GitHub Copilot even for a short period. Once developers have started to use the tool it is hard to imagine being without it.

When discussing GenAI tooling options, the person responsible for AI utilization is interested in understanding what value different kinds of approaches and tooling could bring. For instance, do tools that understand the work context give the most benefits, or would answering single questions give the same benefits. Engineering manager sees by far the most value in tools connected directly to IDE such as GitHub Copilot. According to him, the biggest challenge in improving productivity is understanding where and how to make the changes in the existing large code repository. A tool connected to IDE has the potential to understand the context of the change. Although there are a lot of developers, most development topics have something to do with modifying or extending the existing solutions. Also, most productivity benefits could be gained from directly improving the productivity of programming activities, since most developers can focus on development activities only. Product owners take care of requirements definition.

4.3 Developer GenAI Survey Design

To understand more about the current situation and possible improvement areas in using GenAI a survey was made for developers. The survey included questions about the GenAI tools in use, whether the developers see a positive impact of the tools in their productivity and work experience, the areas where developers use the tools, and improvement areas. The intention was to gather an understanding of how the developers use the tools and how they see that their work and productivity could be further improved. According to affordance theory (Matei, 2020) people find opportunities in digital solutions based on their previous experience. This questionnaire gathers information on what type of opportunities developers have found in using ChatGPT and GitHub Copilot and what possibilities they see available in utilizing the tools in the future. Also, according to the software development productivity theory by Storey et al. (2019), productivity and satisfaction are closely linked. This questionnaire directly asks whether using generative AI tools improves productivity and satisfaction.

The survey was done using the Google Forms tool. Also, the current performance metrics mostly concentrate on aspects such as velocity. The aim was to gather insights about the developer experiences in a similar way as in the SPACE framework. Only a few questions were included since a shorter questionnaire was expected to receive more responses, since filling it takes less effort. The survey was sent to all 40 developers, and it received responses from 20 developers. Table 1 below represents a summary of the questions. In the following sections, the questions and survey answers are discussed in more detail.

Question number	Question	Answer type
1	I use following GenAI tools in daily work	Multiple choice answer
2	Currently used GenAI tools and practices make my work more pleasant	Numerical scale 1-6
3	Currently used GenAI tools and practices make me more productive	Numerical scale 1-6
4	I use GenAI tools in	Multiple choice answer
5	We could use more ways to utilize GenAI in	Multiple choice answer
6	Would you like to try to use some other GenAI tool in development? Which tool?	Open-ended question
7	What should be changed or improved so that we could get more benefits from GenAI?	Open-ended question

Table 1. Summary of survey questions

4.4 Survey Results

4.4.1 GenAI Tool Usage and Experiences

First, it was asked about the GenAI tools used currently in daily work. In the answers, most developers stated using GitHub Copilot (80%). Many also answered using GitHub Copilot chat (40%) and ChatGPT (35%). From more detailed analysis it could be seen that all except one developer use either GitHub Copilot or GitHub Copilot chat. The following Figure 14 shows a summary of the answers.

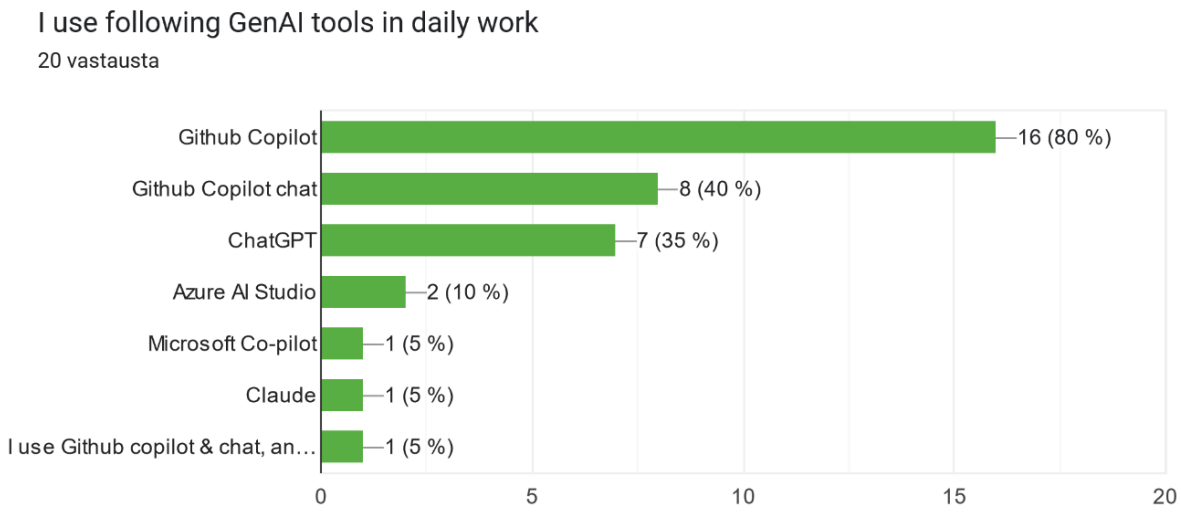

Figure 14. Question 1 answers

The second question was about whether the currently used GenAI tools and practices make work more pleasant. The scale of the answer was from 1 to 6 where 1 meant ‘Disagree completely’ and 6 ‘Agree completely’. From the answers, it can be interpreted that most developers find that the work is more pleasant when using GenAI tools as assistants. The average was 4,35 and the median was 4,5. Results were similar for both groups, those that use only GitHub Copilot (Chat) and those that also use ChatGPT. The following figure 15 shows a summary of the answers to this question.

Picture 15 Question 2 answers

The third question was about whether the currently used GenAI tools and practices make developers more productive from their perspective. Here also the results are positive, since most developers evaluate that the tools make them more productive. The average was 4.4 and the median was 4.5. The results were very similar from this question to the results from work satisfaction improvements. Results were also similar for both groups, those that use only Github Copilot (Chat) and those that also use ChatGPT Figure 16 shows the summary of the answers.

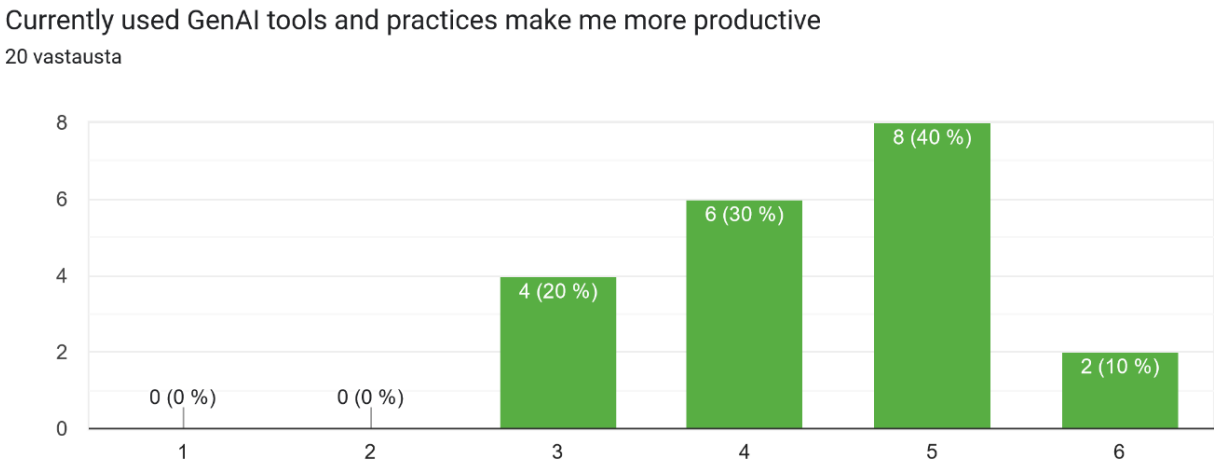

Figure 16. Question 3 answers

The last section in this area was about in which areas the developers currently use GenAI tools. In case company the developers use GenAI tools for many different types of tasks. The most common tasks are Code generation (90%), ideating solutions/concepts (50%), testing (50%) and documentation (50%). Also commenting (40%) and code review (25%). Some individual users also use the tools for architectural work, Fixing errors, debugging, re-phrasing sentences, and getting an idea of what to search for. The figure 17 shows the statistics for this question.

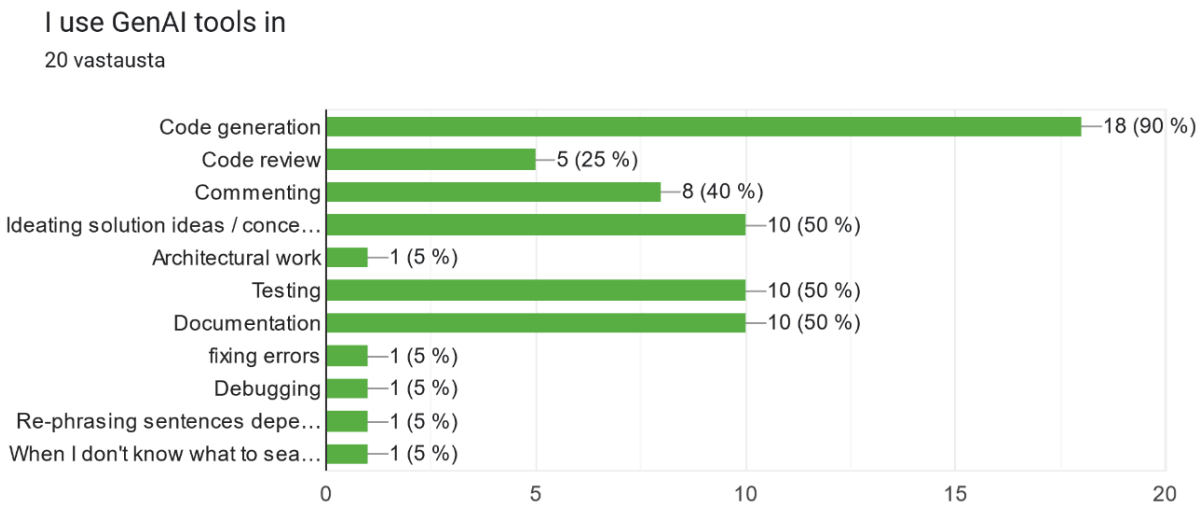

Figure 17. Question 4 answers

The data about the used tools and usage areas together shows clearly that ChatGPT users tend to be the ones who use the tools for ideating solutions/concepts. All ChatGPT users state that they use the tools to ideate solutions/concepts. Also, another user who did not use ChatGPT stated to us the tools for this purpose while using GitHub Copilot and Azure AI Studio.

4.4.2 Improvement Areas in GenAI adoption

It was asked by developers whether there could be more ways to utilize GenAI. The most common answers were documentation (65%), code review (60%) and testing (50%). Also, there were answers about code generation (40%), commenting (40%), and ideating solution ideas/concepts (30%). Also, architectural work and dependency management got some individual answers. A few thought the teams were already on a sufficient level. The figure 18 shows the statistics for this question.

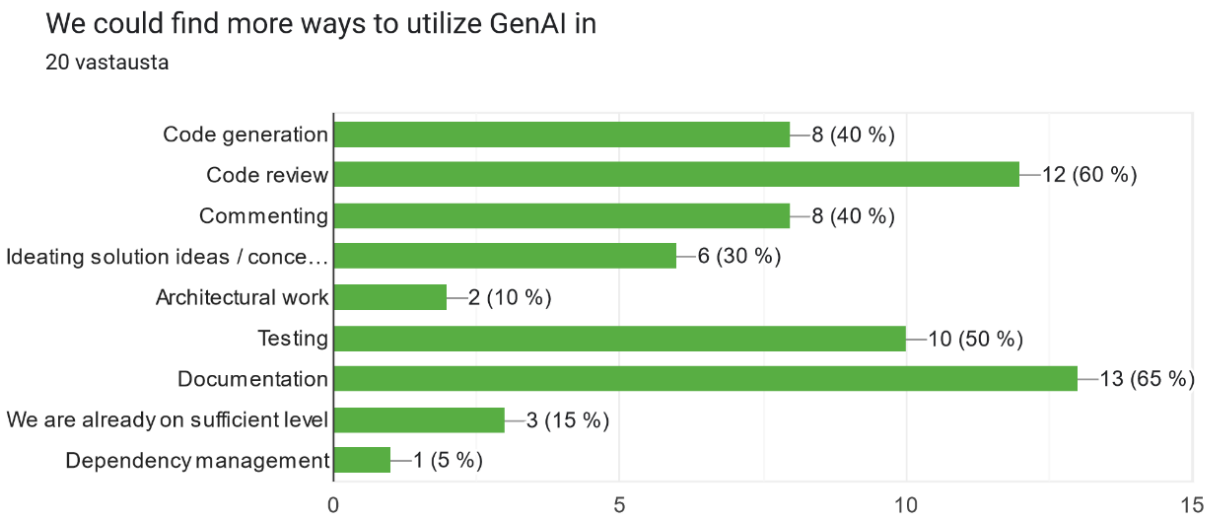

Figure 18. Question 5 answers

The next question was about whether the developers would like to try some other GenAI tool in development and which tool. This was an open-ended and not compulsory question. It received 6 answers out of which half stated that they would not like to try out any other tool specifically. An example of an answer was: “Not currently. Current tools are good enough.”. One person was willing to try GitHub Co-Pilot Enterprise, and one wanted to try out the GenAI tool Cursor with the answer “More advanced and better-integrated tooling e.g. Cursor”.

The last question was about what should be changed or improved so that most benefits could be gained from AI. 9 people answered this open-ended question. A couple of the answers were related to improving the quality of the output of the GenAI tools. Other answers varied in different areas. One person suggested improving the prompting skills by buying good courses for developers. Also, someone else suggested exposing developers more to these tools through training and workshops. One person summarized thoughts like this: “I think it's a highly personal thing depending on the developer, some tools work better for others and some people know how to use e.g. Gen AI tools more efficiently.”

5 Discussion

The goal of this thesis was to answer how generative AI solutions GitHub Copilot can be used to improve software development productivity. The main research questions were stated as follows:

- RQ1: What are the key differences in using GitHub Copilot and ChatGPT for improving software development productivity?
- RQ2: Which type of assistance brings the most productivity improvements?
- RQ3: How can the productivity improvements gained from using GenAI tools in software development be measured?

The thesis contained a literature review on the tools, usage of generative AI tools, and software development productivity. Also, related theoretical frameworks were included. The case study was made based on the theories and to deepen understanding of the way developers currently use the tools and how it could be improved to gain productivity improvements. The results of the research questions are analyzed in the following sections.

5.1 RQ1 Key differences in using GitHub Copilot and ChatGPT

The literature gave good background information on the tools and their differences. The differences between the tools are quite clear as GitHub Copilot is a tool connected to IDE and ChatGPT is a chat-based tool used outside of the programming environment. Also, the tools have been trained on different data, as ChatGPT has been trained on text found through search engines and GitHub Copilot has been trained on a large amount of source code. Also, GitHub Copilot has a chat, but it is limited to programming questions (GitHub, About GitHub Copilot chat). For this reason, the tools support different kinds of use cases. GitHub Copilot is well suited for use in acceleration mode, where the developer already understands the problem to be solved (Liang et al., 2023). ChatGPT can be used in the ideation and architectural design phase in addition to programming tasks (Hao et al., 2024). However, as it is not connected to IDE it has limited understanding of the context.

Also, the way that developers interact with the tools is different. With ChatGPT, the developer needs to be aware of prompting techniques to gain the most out of the tool (AlOmar et al., 2024). Also, the developer should be aware of the possibilities that the tool has to offer at different phases of the development process. As described in affordance theory (Matei, 2020), digital tools get their meaning from the person's experience. This can also be true in the context of using generative assistants for software development, as there are various use cases to which the tool could be applied. Based on the research and the case study the developers use the tools for a variety of purposes. The variety of different use cases can be especially true for ChatGPT as it is not intended only as a programming assistant.

In the case study, it was noted that because the teams have a large existing code base and most development some type of further development, an understanding of the context is a very important aspect of the GenAI tool. GitHub Copilot has already developed in the direction of having a chat interface so those that enjoy that type of interaction can also use that tool. It appears that many developers have already taken the chat functionality into use. However, many developers in the survey stated that they use both tools GitHub Copilot and ChatGPT in their daily work. Even in this environment, these different types of tools seem to complement each other at least for part of the developers.

5.2 RQ2 Which Type of Assistance Brings the Most Productivity Improvements

As described in the research section, development can be described as a process with interactions between stages. These stages cannot be considered in isolation and understanding of productivity must be based on understanding how developers spend their time. (Meyer et al., 2018) This means that also productivity improvements with GenAI tools need to be looked at from this perspective. A specific method or tool taken into use may not be a silver bullet if the solution does not help with those tasks that take most of the developer's time.

There were individual studies for measuring the performance improvements with both tools. However, the scientific evidence was not yet strong on the capabilities of either of the tools to provide measurable productivity improvements. More abstract challenges may also not be solvable for the solutions although it is likely to be possible in the future (Pudari et al., 2023). According to the software development productivity theory by Storey et al. (2019), the satisfaction and productivity of developers are closely linked and have a bi-directional impact. According to the literature section developers have taken the GenAI solutions into use very actively and most feel that they improve productivity (AlOmar et al., 2024), so this would imply the assistants help software developers in their work which leads to better productivity. Also, in the case study company, most developers felt positive about the tools improving their work satisfaction and productivity. The positive impact was true for both ChatGPT and GitHub Copilot users. Based on these results generative AI assistants, as part of the engineering system, could improve both satisfaction and productivity of developers.

The purpose of using GitHub Copilot is most often related to getting help in generating code (Zhang et al. 2023, 10). In programming assignments comparing different tools GitHub Copilot Chat was the strongest performer (Agarwas et al., 2024). The overall results would indicate that GitHub Copilot or GitHub Copilot chat are strong in improving productivity in purely programming-related tasks since they are also directly available in IDE.

According to the analysis, developers seek assistance from ChatGPT across 16 types of inquiries. In addition to programming questions, there are conceptual enquiries and how-to questions. (Hao et al., 2024) As using ChatGPT is based on natural language interactions, it can help with a wide range of topics. This could indicate that if the bottleneck in improving productivity is somewhere else than in the actual programming phase, ChatGPT or some other tool could provide more help than GitHub Copilot.

Setting up proper targets that aim to measure different aspects of productivity such as with the SPACE framework (Forsgren et al., 2021) could help in identifying the bottlenecks and selecting appropriate approaches and generative AI tooling to solve them. What needs to be considered is the readiness of developers to take up GenAI solutions into use, as only a

solution that is actively used can provide value. As implied by the technology acceptance model (Marikvan et al.), people consider both the ease of use and usefulness of the solutions before taking them into daily use. Also as described in the theory of planned behavior a person with strong intention and confidence in mastering a subject is most likely to preserve (Ajxen. 1991). With some tools such as ChatGPT, developers could need some additional guidance in finding the easiest and most beneficial ways to utilize the tools. This could be especially true with GenAI tools that are not designed for one specific use case and require learning prompting techniques.

.

Based on the case company survey, the GenAI tools were used most often for code generation, but also quite often for documentation, testing and ideating. The most used tools were GitHub Copilot, GitHub Copilot Chat and ChatGPT. According to interviews with management, GitHub Copilot was the tool that developers were most encouraged to use. It appears that some developers have turned to ChatGPT for some additional use cases.

According to the survey the developers see opportunities in utilizing AI more in many areas such as documentation, testing and review. For example, code documentation is an area where Microsoft Copilot and ChatGPT outperform GitHub Copilot Chat since it cannot provide longer or in-depth explanations in plain English (Agarwas et al., 2024). If lacking documentation or some other areas are productivity bottlenecks it might be useful to systemically evaluate which tools support these use cases. Based on the survey, developers in the case company were quite happy with the currently used tools. One open-ended comment was thoughtful: “I think it's a highly personal thing depending on the developer, some tools work better for others and some people know how to use e.g. Gen AI tools more efficiently.” This comment may very well be true that it is a highly personal thing how developers can incorporate the tools in their workflow. Still, it might be beneficial to share the learnings systemically from time to time, as people find different opportunities in tools as implied by affordance theory. For example, part of the developers used ChatGPT. Have other developers found the same opportunities in using this tool?

5.3 RQ3 How to Measure Productivity Improvements

Measuring productivity with more than only one metric helps to create a better understanding of productivity so better decisions can be made for improvement. For instance, measuring activity counts alone does not tell the reason behind the improvement. (Forsgren et al., 2021) However, activity count is the best predictor of perceived productivity. (Ziegler et al., 2022) Since software development productivity cannot be defined by one single definition and metric, it is best to measure against different metrics that are important within the context. According to software development theory (Storey et al. 2019) many social and technical factors affect developer satisfaction and productivity. SPACE framework is one concrete tool to measure developer productivity (Forsgren et al., 2021).

In the case study company, the overall conclusion of the management was that the GenAI tools improve productivity. Productivity numbers had been improving and programmers would not give up on the tools. To complement this knowledge, two survey questions were included to understand the satisfaction and productivity improvements that developers experience. Productivity and satisfaction are expected to be also closely linked to each other (Storey et al., 2019), so this gives a more complete understanding of the improvement. The developers were asked first whether the currently used GenAI tools and practices make their work more pleasant and then whether they make them more productive. The averages and medians for both questions were positive, so this implies that the tools give productivity benefits. Complementing the programming efficiency metrics with other data gives a more complete picture of the level of improvement. In the future, some system-level metrics such as (internal) client satisfaction could also be incorporated.

5.1 Limitations

At the time of writing this thesis in 2024 the interest and availability of generative AI programming assistants is still new. This is reflected in the amount of available research and for instance, the studies comparing the two tools or their impact on productivity

improvements were still scarce. Also, the tools are rapidly involved so the research done last year may already be partly outdated, for example, only a few sources mentioned GitHub Copilot chat which is a new feature in GitHub Copilot. The case study was limited to the experiences of one organization and their developers so a wider audience, and their experiences were not studied.

5.2 Future Research

A future research topic could be a controlled study with two groups implementing the same project with another group using GitHub Copilot and another with the help of ChatGPT. The results in metrics such as speed of development and number of errors could be combined with the experiences of developers. This study would deepen the understanding of how the two tools compare to each other in improving productivity. Also, a future research topic would be to understand better the motivational factors and experiences that developers have in using a specific solution. This would also help in designing tools that are better incorporated into the entire workflow from design to deployment.

6 Conclusion

The objective of the study was to investigate how generative AI solutions GitHub Copilot and ChatGPT can be used to improve software development productivity. The differences between the tools were investigated especially in the context of productivity. Also measuring productivity gains was researched.

Generative AI tools show great potential in improving the productivity of software development. Developers have quite widely taken the tools into use. Currently, software developers get clear benefits from the assistants in code generation, especially in repetitive tasks. More abstract challenges may not always be solvable for the assistants, but the tools are on their way to being able to also support these cases. Software development productivity and satisfaction may have a bi-directional impact, and generative AI assistants show the potential to improve both aspects. Different GenAI tools may be most helpful in

separate areas of software development depending on for example the source data they have been trained on. As implied by affordance theory, developers may find different opportunities in using the tools based on their background, so also the productivity benefits may vary.

It was concluded that GitHub Copilot is well suited to be used in acceleration mode when the developer already understands the problem to be solved. ChatGPT can be used in the ideation and architectural design phase in addition to programming tasks. However, as ChatGPT is not connected to IDE it has limited understanding of the context. Evaluating productivity needs to be based on understanding how developers spend their time. Also, productivity improvements with GenAI tools need to be evaluated from this perspective as the solution needs to help in those task that take the developer's time.

The overall results indicate that GitHub Copilot or GitHub Copilot chat is stronger in improving productivity in directly programming related tasks, especially since the tools are connected to IDE. Using ChatGPT is based on natural language interactions and can help in a wide range of areas such as conceptual inquiries. In the end, which tool brings the best benefits is context-dependent and the willingness and ability of the developer to use a specific tool is an important aspect.

Software development productivity is best measured with multiple relevant metrics that are important within the context. The acceptance rate is the best predictor of productivity when measuring productivity improvements using GitHub Copilot. Including measurements for example for self-perceived productivity and work satisfaction improvements from GenAI tools gives a more complete picture of the direct benefits of the use of assistants.

Generative AI assistants are already supporting developers in many ways to be productive in daily work. The solutions will likely continue to improve and ways to incorporate the tools into the workflow of developers will get better. Separate Gen AI tools such as GitHub Copilot and ChatGPT support partly different use cases. It is recommended that organizations and developers explore the options to find solutions that suit their targets and preferences.

7 References

Agarwal, V., Garg, M., Dharmavaram, S., Kumar, D. 2024. "Which LLM should I use?": Evaluating LLMs for tasks performed by Undergraduate Computer Science Students. Conference on International Computing Education Research. https://doi.org/10.48550/arXiv.2402.01687

Ajzen, I. 1991.The theory of planned behavior. Organizational Behavior and Human Decision Processes. Volume 50 Issue 2, Pages 179-211. https://doi.org/10.1016/0749-5978(91)90020-T

AlOmar, E A., Venkatakrishnan, A., Mkaouer, M W., Newman, Christian, D., Ouni, A. 2024. How to Refactor this Code? An Exploratory Study on Developer-ChatGPT Refactoring Conversations. Proceedings of MSR '24: Proceedings of the 21st International Conference on Mining Software Repositories. 6 pages. https://doi.org/10.48550/arXiv.2402.06013

Fan, A., Gokkaya, B., Harman, M., Lyubarskiy, M., Sengupta, S., Yoo, S., Zhang, J. 2023. Large Language Models for Software Engineering: Survey and Open Problems. https://doi.org/10.48550/arXiv.2310.03533

Forsgren, M., Storey, M-A, Maddila, C., Zimmermann, T. Brian, H., Butler, J. 2021. The SPACE of Developer Productivity.Queue, Volume 19, Issue 1. https://doi.org/10.1145/3454122.3454124

GitHub. About GitHub Copilot. https://docs.GitHub.com/en/copilot/about-GitHub-copilot

GitHub. About GitHub Copilot chat. https://docs.GitHub.com/en/copilot/GitHub-copilot-chat/about-GitHub-copilot-chat

GitHub. About GitHub Copilot chat in your ide. https://docs.GitHub.com/en/copilot/GitHub-copilot-chat/copilot-chat-in-ides/about-GitHub-copilot-chat-in-your-ide

Hao, H., Kazi, A., Qin, H., Macedo, M., Tian, Yuan., Ding, S., Hassa, A. 2024. An Empirical Study on Developers' Shared Conversations with ChatGPT in GitHub Pull Requests and Issues. https://doi.org/10.48550/arXiv.2403.10468

Hevner, A., March, S., Jinsoo, P. Design science in information systems research. 2004. https://www.jstor.org/stable/25148625

Kailun, J., Chung-Yu, W., Hung, P., Hemmati, H. 2024. Can ChatGPT Support Developers? An Empirical Evaluation of Large Language Models for Code Generation. MSR '24: Proceedings of the 21st International Conference on Mining Software Repositories. https://doi.org/10.48550/arXiv.2402.11702

Kalliamvakou, E. 2022**.** Research: quantifying GitHub Copilot's impact on developer productivity and happiness. https://GitHub.blog/2022-09-07-research-quantifying-GitHub-copilots-impact-on-developer-productivity-and-happiness/

Komp-Leukkunen, K. 2024. How ChatGPT shapes the future labour market situation of software engineers: A Finnish Delphi study. Futures volume 160. https://doi.org/10.1016/j.futures.2024.103382

Koul, S. Eydgahi, Al. 2017 A systematic review of technology adoption frameworks and their applications. Journal of Technology Management & Innovation vol.12 no.4. http://dx.doi.org/10.4067/S0718-27242017000400011

Lawton, G. 2023. GitHub Copilot vs. ChatGPT: How do they compare? https://www.techtarget.com/searchenterpriseai/tip/GitHub-Copilot-vs-ChatGPT-How-do-they-compare

Liang, T., Yang, C., Myers, B. 2023. A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges. ICSE '24: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering. Article No.: 52, Pages 1 – 13. https://doi.org/10.1145/3597503.3608128

Liukko, V., Knappe, A., Anttila, T., Hakala, J,. Ketola, J., Lahtinen, D., Poranen, T., Ritala, T-M., Setälä, M., Hämäläinen. H., Abrahamsson, P. 2024. ChatGPT as Full-Stack WebDeveloper. P 2022/2023 Workshops, LNBIP 489, pp. 201–209- https://urn.fi/URN:NBN:fi:tuni-202401151431

Mandvikar, S. 2023. Factors to Consider When Selecting a Large Language Model: A Comparative Analysis. International Journal of Intelligent Automation and Computing, 6(3), 37–40. https://research.tensorgate.org/index.php/IJIAC/article/view/53

Mann, K., O'Connor, F. 2023. Gartner Research: How to Use Generative AI to Boost Developer Productivity. https://www.gartner.com/en/software-engineering/insights/how-to-use-gen-ai-to-boost-developer-productivity

Marikvan, D., Papagiannidis, S. Tehcnology acceptance model (TAM). TheoryHub Book. ISBN: 9781739604400. https://open.ncl.ac.uk/theories/1/technology-acceptance-model/

Matei, S. 2020. What is affordance theory and how can it be used in communication research? https://doi.org/10.48550/arXiv.2003.02307

Mathieson, 1991. Predicting User Intentions: Comparing the Technology Acceptance Model with the Theory of Planned Behavior. Information Systems Research 2(3). DOI:10.1287/isre.2.3.173

McKinsey Digital. 2023. Unleashing developer productivity with generative AI. https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/unleashing-developer-productivity-with-generative-ai

Melegati, J., Guerra, E. 2023. DAnTE: a taxonomy for the automation degree of software engineering tasks. https://doi.org/10.48550/arXiv.2309.14903

Meyer, A., Barr, E., Bird, C., Zimmermann, T. 2018. Today was a Good Day: The Daily Life of Software Developers. https://discovery.ucl.ac.uk/id/eprint/10090507/1/Barr_Today%20was%20a%20Good%20Day.%20The%20Daily%20Life%20of%20Software%20Developers_AAM2.pdf

Moradi Dakhel, M., Majdinsasab, V., Nikanjam, A., Khomh, F., Desmarais, M., Jiang, Z. 2024. GitHub Copilot AI pair programmer: Asset or Liability? Journal of Systems and Software, Volume 203. https://doi.org/10.1016/j.jss.2023.111734

Nguyen-Duc, A., Cabrero-Daniel. Przybyle, A., Arorae, C., Khanna, D., Herda, T., Rafiq, U., Melegati., J., Guerra, E., Kemell, K., Saari, M., Zhangim Z, Huy, Lej,, Quanj, T., Abrahamsson, P. 2023. Generative Artificial Intelligence for Software Engineering - A Research Agenda. https://doi.org/10.48550/arXiv.2310.18648

Nguyen-Duc, A., Abrahamsson, P., Khomh, F. 2024. Generative AI for Effective Software Development. Springer.

Ouyang, S., Zhang, J., Harman, M., Wang, M. 2023. An Empirical Study of the Non-determinism of ChatGPT in Code Generation https://doi.org/10.48550/arXiv.2308.02828

Pudari, R., Ernst, N. 2023. From Copilot to Pilot: Towards AI Supported Software Development. https://doi.org/10.48550/arXiv.2303.04142

Pereira, G., Prikladnicki, R., Jackson, V., Hoek, A., Fortes, L., Macaubas, I. 2024. Early Results from a Study of GenAI Adoption in a Large Brazilian Company: The Case of Globo. https://doi.org/10.1007/978-3-031-55642-5_13

Rosenkilde, J. 2024. How GitHub Copilot is getting better at understanding your code.

https://GitHub.blog/2023-05-17-how-GitHub-copilot-is-getting-better-at-understanding-your-code/

Runeson, P., Höst, M., Rainer, A. Regnell, B. Case study research in software engineering, guidelines and examples. 2012. Wiley.

Sansom, R. 2021. Theory of Planned Behavior. https://ascnhighered.org/ASCN/change_theories/collection/planned_behavior.html

Storey, M-A., Zimmermaan, T., Bird, C., Cxerwonka, J., Murphy, B., Kalliamvakou, E. 2019. 2019. Towards a Theory of Software Developer Job Satisfaction and Perceived Productivity**.** IEEE Transactions on Software Engineering. DOI: 10.1109/TSE.2019.2944354

Ulfnes, R., Moe, N. Stray, V., Skarpen, M. 2024. Transforming Software Development with Generative AI: Empirical Insights on Collaboration and Workflow. https://doi.org/10.48550/arXiv.2405.01543

Vaithilingam, P., Zhang, T., Glassman, E. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. CHI EA '22: CHI Conference on Human Factors in Computing Systems Extended. Article No.: 332, Pages 1 – 7. https://doi.org/10.1145/3491101.3519665

Verner, J., Sampson, V., Tosic N, Bakar, A., Kitchenham, A. Guidelines for industrially based multiple case studies in software engineering. 2009. DOI:10.1109/RCIS.2009.5089295

White, J., Hays, S., Ouchen, F., Spencer-Smith, J., Schmidt, C. 2023. ChatGPT Prompt Patterns for Improving Code Quality, Refactoring, Requirements Elicitation, and Software Design. https://arxiv.org/abs/2303.07839

White, J., Fu, O., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., Schmidt, D. A 2023. Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. https://doi.org/10.48550/arXiv.2302.11382

Yetiştiren, B., Özsoy, I., Ayerdem., M., Tüzün, E. 2023. Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. https://doi.org/10.48550/arXiv.2304.10778

Zhang, B., Liang, P., Zhou, X., Ahmad, A., Waseem, M. 2023. Demystifying Practices, Challenges and Expected Features of Using GitHub Copilot. International Journal of Software Engineering and Knowledge Engineering. https://doi.org/10.48550/arXiv.2309.05687

Ziegler, A., Kalliamvakou, E., Alice L, X., Rice, A., Rifkin, D., Simister, S., Sittampalam, G., Aftandilian, E. 2022. Productivity Assessment of Neural Code Completion. Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming (MAPS ’22). https://dl.acm.org/doi/pdf/10.1145/3520312.3534864